



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



SOICT

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Lecture 2.2: Unimodal Representations (Part 1)

Dam Quang Tuan



Lecture Objectives

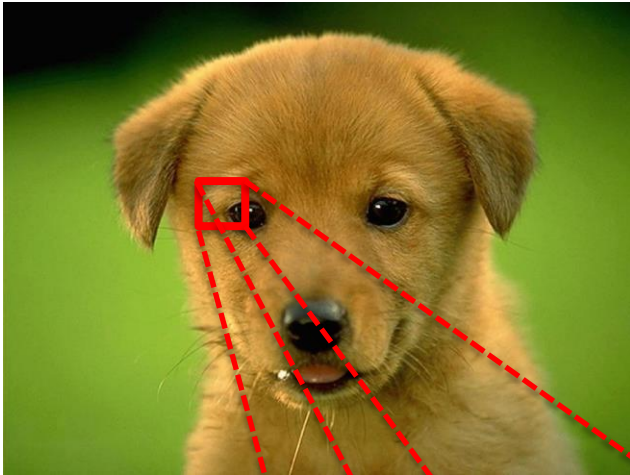
- Unimodal basic representations
- Dimension of heterogeneity
- Image representations
 - Image gradients, edges, kernels
- Convolution neural network (CNN)
 - Convolution and pooling layers
- Visualizing CNNs
- Region-based CNNs

Unimodal Basic Representations



Unimodal Representation – Visual Modality

Color image



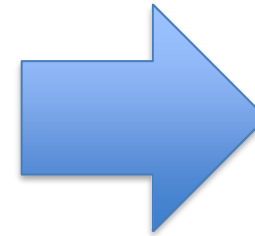
Each pixel is represented in $\mathcal{R}^d$, d is the number of colors ($d=3$ for RGB)

88	82	84	88	85	83	80	93	102
88	80	78	80	80	78	73	94	100
85	79	80	78	77	74	65	91	99
38	35	40	35	39	74	77	70	65
20	25	23	28	37	69	64	60	57
22	26	22	28	40	65	64	59	34
24	28	24	30	37	60	58	56	66
21	22	23	27	38	60	67	65	67
23	22	22	25	38	59	64	67	66

Input observation x_i

88
88
85
38
20
22
24
21
23
82
80
79
35
25
26
28
22
22
84
78
80
⋮

Binary classification problem



Dog ?

label $y_i \in \mathcal{Y} = \{0,1\}$

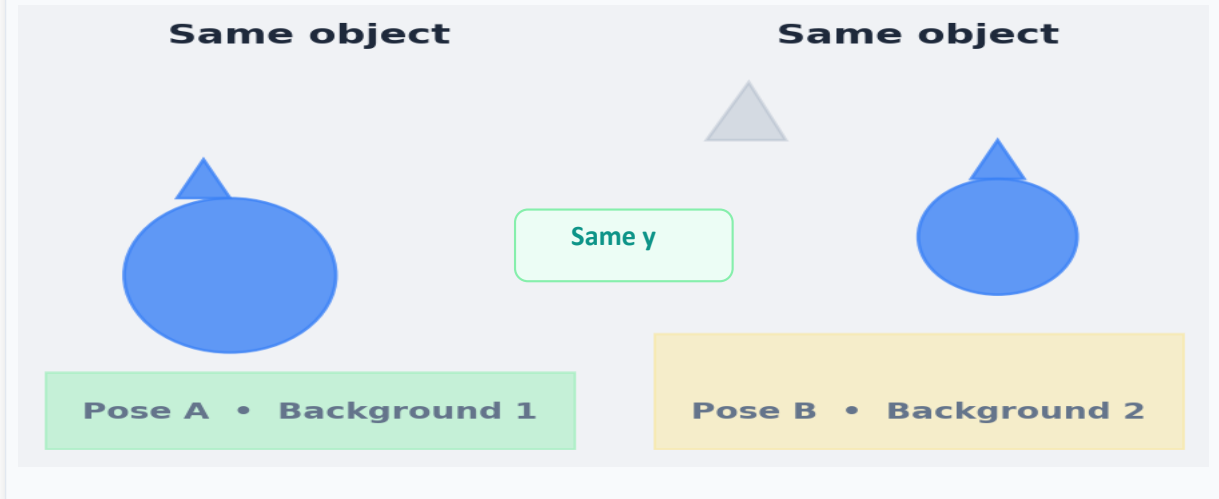
Key points

- Representation for images is typically a tensor of pixel values (often normalized).
- For RGB: $X \in \mathbb{R}^{H \times W \times 3}$.
- Binary label: $y \in \{0,1\}$.
- Pitfall: the same class can look very different under changes in lighting, scale, or background.

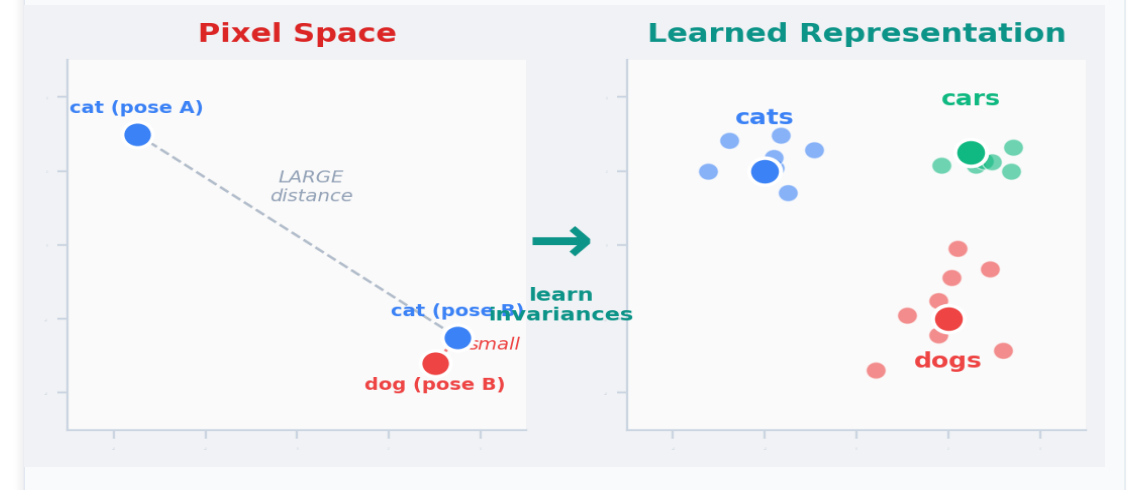
Pixels Are Not Semantics

Why Invariances Matter for Representation Learning

The Problem



The Solution



↔ **Core problem:** Raw pixels can change dramatically while the semantic label stays the same. Conversely, different objects in the same pose can have very similar pixel vectors. Pixel distance $\neq$ semantic similarity.

Data Augmentation

Forces the model to see the same object under different transformations — crops, flips, color jitter, rotations. The model learns that these variations don't change the label.



Convolution + Pooling

Convolution gives local weight sharing; pooling provides local translation invariance by design. Together they build representations that are robust to small spatial shifts.



Attention Mechanisms

Transformers learn to focus on relevant parts regardless of position. Self-attention makes the representation permutation-equivariant — location becomes a learned property.

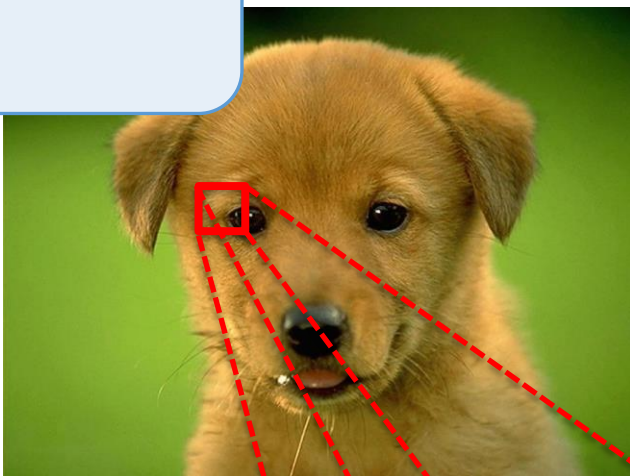
For GenAI: When a VAE or diffusion model learns a latent space, it compresses away nuisance variations and keeps semantic structure — that's what makes the latent space smooth enough to generate from.

Unimodal Representation – Visual Modality

If T appears elsewhere:

T = sequence length or number of tokens

Example: $224 \times 224 \times 3$



Each pixel is represented in $\mathcal{R}^d$, d is the number of colors ($d=3$ for RGB)

88	82	84	88	85	83	80	93	102
88	80	78	80	80	78	73	94	100
85	79	80	78	77	74	65	91	99
38	35	40	35	39	74	77	70	65
20	25	23	28	37	69	64	60	57
22	26	22	28	40	65	64	59	34
24	28	24	30	37	60	58	56	66
21	22	23	27	38	60	67	65	67
23	22	22	25	38	59	64	67	66

Input observation x_i

88
88
85
38
20
22
24
21
23
82
80
79
35
25
26
28
22
22
84
78
80
⋮

**Multi-class
classification problem**

Duck

-or-

Cat

-or-

Dog

-or-

Pig

-or-

Bird ?

label $y_i \in \mathcal{Y} = \{0,1,2,3, \dots\}$

Inductive Biases for Vision

- **Locality**

- Nearby pixels are usually more related than distant pixels, so small filters capture useful local patterns such as edges and corners.

- **Translation equivariance**

- The same feature matters no matter where it appears in the image, so shared filters can detect it across positions.

- **Hierarchical composition**

- Lower layers learn edges and textures, middle layers learn parts, and deeper layers combine them into object-level concepts.

These biases make vision models more data-efficient and more effective on images.

Unimodal Representation - Language Modality

Written language

★★★★★ **Masterful!**

By Antony Witheyman - January 12, 2006

Ideal for anyone with an interest in disguises who likes to see the subject tackled in a **humorous** manner.

0 of 4 people found this review helpful

poken language

MARTHA (CON'T)
Look around you. Look at all the
great things you've done and the
people you've helped.

CLARK
But you've only put up the good
things they say about me.

MARTHA
Clark, honey. If I were to use the
bad things they say I could cover
the barn, the house and the
outhouse.

[illegible]

“one-hot” vector

Word-level classification

Part-of-speech ?

Sentiment ?

Named entity ?

Pipeline:



ĐẠI HỌC BÁCH KHOA HÀ NỘI **Te**
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Text

Tokenize

IDs

Embedding lookup

**Sequence of
vectors ($T \times d$)**

written or spoken input

Language Modality

From Text to Numbers — Tokenization. One-Hot. and Embeddings

A

Pipeline:

Raw Text
"The cat sat..."



Tokenize
BPE / SentencePiece



Token IDs
[101, 2543, ...]



One-Hot
dim = $|V|$ (sparse)

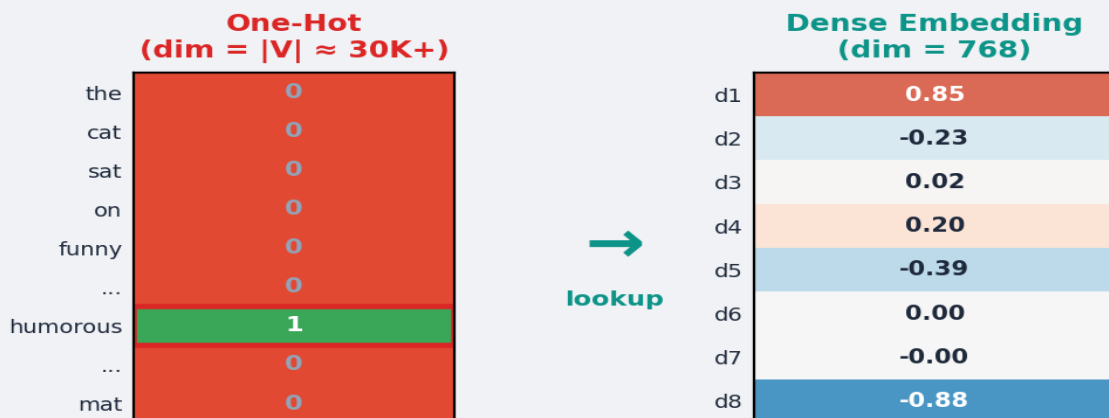


Embedding
lookup (dense)



$T \times d$ matrix
sequence repr.

One-Hot → Dense Embedding



Why Embeddings Matter

Embedding Space: Similar Words → Nearby Vectors



Tokenization

Modern models (GPT, BERT) use subword tokenization — BPE or SentencePiece. Handles rare words and morphology better than word-level. Vocab size: 30K–100K tokens.

From Sparse to Dense

One-hot: dim = $|V|$, no similarity info.
Embedding lookup: dense vector in $\mathbb{R}^d$ ($d = 768$). Now "humorous" and "funny" can be nearby. Output: $T \times d$ matrix for sequence.

Downstream Tasks

POS tagging, sentiment analysis, NER — drove early representation learning. Sentiment requires whole-sentence understanding → sequence models: RNNs, LSTMs, Transformers.

GPT connection: GPT uses exactly this pipeline — text → tokens → IDs → embeddings → Transformer → predict next token. The representation is identical; only the training objective differs (generation vs. classification).

Tokenization

Goal: convert raw text into manageable units before mapping them to IDs and embeddings.

Word tokenization

Example

"Masterful humourous manner"

["Masterful", "humourous", "manner"]

- simple and intuitive
- every unseen word can become OOV

Subword tokenization

Example

"unhappiness"

["un", "happi", "ness"]

- smaller shared vocabulary
- rare words can reuse known pieces

Why subword helps OOV

- handles unseen or rare words
- captures prefixes, suffixes, and morphology
- reduces vocabulary size
- works better with misspellings and names

Modern tokenizers often use BPE, WordPiece, or SentencePiece.

Unimodal Representation – Language Modality

Written language



Masterful!

By Antony Witheyman - January 12, 2006

Ideal for anyone with an interest in disguises who likes to see the subject tackled in a humorous manner.

0 of 4 people found this review helpful

Spoken language

MARTHA (CON'T)

Look around you. Look at all the great things you've done and the people you've helped.

CLARK

But you've only put up the good things they say about me.

MARTHA

Clark, honey. If I were to use the bad things they say I could cover the barn, the house and the outhouse.

Input observation x_i

0
1
0
0
1
0
1
0
0
0
0
1
0
0
0
0
1
0
0
0
⋮

“bag-of-words” vector

$|x_i|$ = number of words in dictionary

Document-level
classification

Sentiment ?
(positive or negative)

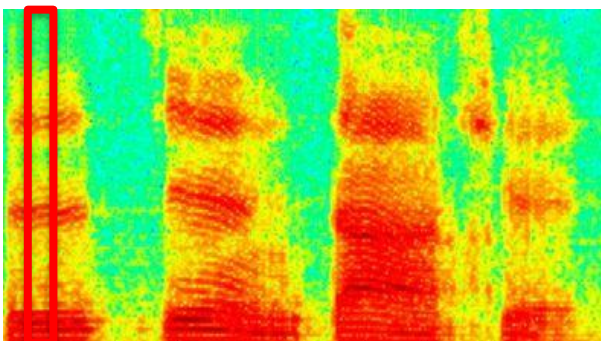
What happens with word ordering?

Unimodal Representation - Acoustic Modality

Digitalized acoustic signal



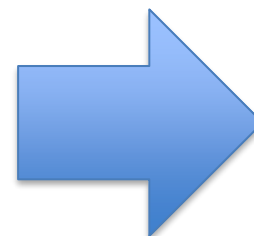
- Sampling rate: 8-96 kHz
- Bit depth: 8, 16, or 24 bits
- Window length: 20 ms
 - Hop size: 10 ms



Spectrogram

Input observation x_i

0.21
0.14
0.56
0.45
0.9
0.98
0.75
0.34
0.24
0.11
0.02



Spoken word ?

Feature extraction pipeline

Waveform -> windowing -> FFT -> spectrogram

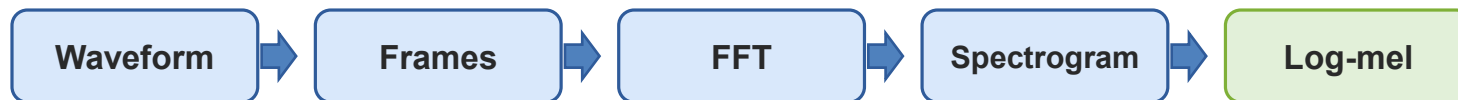


Each short window yields one spectrum; stacking them forms a spectrogram.

Unimodal Representation - Acoustic Modality

STFT terms

STFT = Short-Time Fourier Transform: compute a spectrum for each short analysis window.



Window length

Duration of each analysis frame.
Typical speech setting: 20-25 ms.
Longer windows improve frequency resolution.

Hop size

Shift between adjacent frames.
Typical speech setting: 10 ms.
Smaller hops give smoother time tracking.

Frequency bins

FFT outputs magnitudes at discrete frequencies.
More FFT points provide finer frequency resolution.

Why log-mel is common

The mel scale better matches human pitch perception.
Log compression reduces dynamic range and highlights useful patterns.
It is lower-dimensional and often less noisy than a raw linear spectrogram.
Log-mel features are a standard input for speech recognition and audio classification.

Common speech setup: frame length 25 ms, hop 10 ms, then mel filtering + log.

Unimodal Representation - Acoustic Modality

Waveform vs spectrogram

Two common input forms. Choose the one that best exposes the patterns your model needs.

Waveform

Raw amplitude samples over time

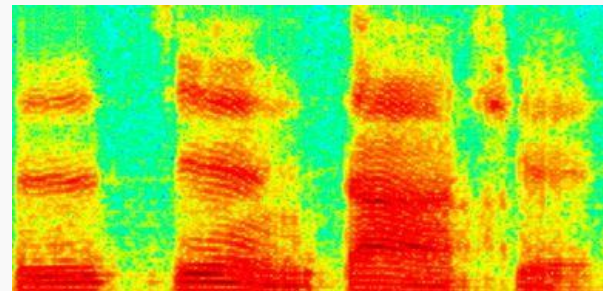


$x \text{ in } \mathbb{R}^T$

- Raw samples with exact temporal detail
- Use when fine timing cues matter
- Common in end-to-end audio and generation

Spectrogram / log-mel

Time-frequency representation



$X \text{ in } \mathbb{R}^{(F \times T)}$

- F = frequency bins, T = time frames
- Highlights harmonics and energy patterns
- Common in speech/audio classification

Rule of thumb: start with spectrogram / log-mel for recognition; use waveform when raw timing details are crucial or you want end-to-end feature learning.

Unimodal Representation - Acoustic Modality

Audio invariances

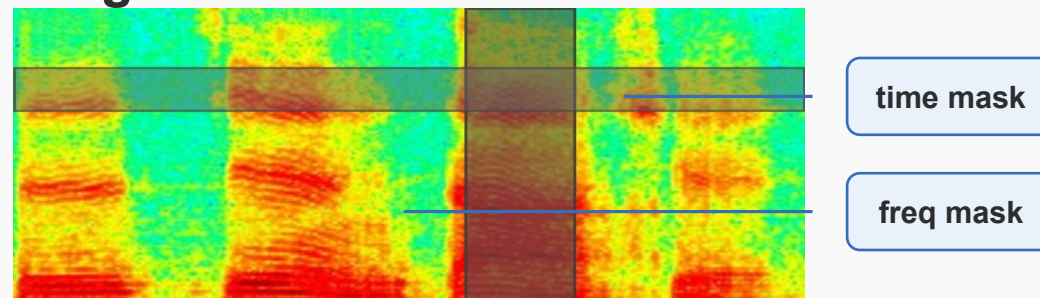
Good representations should be stable to nuisance changes while preserving task-relevant cues.

Useful invariances

- Small shifts in time or onset
- Mild background noise
- Small loudness or gain changes
- Minor channel / microphone changes

Task matters: for emotion recognition, prosody and pitch are informative, so we should not remove all variability.

Augmentation



Common

time shift

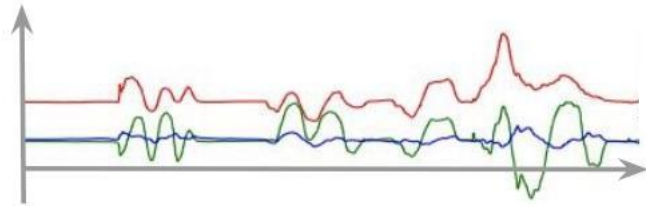
time mask

freq mask

noise

Goal: ignore nuisance variability, but keep the information that predicts the downstream label.

Unimodal Representation – Sensors



Six-axis force-torque
sensor:
 $\mathbf{X} \in \mathbb{R}^{T \times 6}$

Multivariate time series

$$\mathbf{X} \in \mathbb{R}^{T \times d}$$

T = time steps, d = channels

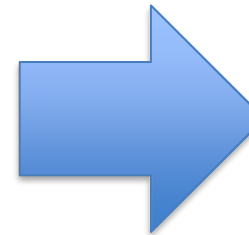
Force-Torque Sensor



Proprioception

Examples: IMU, robot joint angles /
velocities, ECG.

Position / velocity
stream:
 $\mathbf{X} \in \mathbb{R}^{T \times 2d}$



Next action

Resampling + normalization for sensors

Resampling / synchronization

Align sensor streams to a common timestamp grid.

Resample to a shared rate when sensors run at different frequencies.

Interpolate, forward-fill, or mask missing values before modeling.

Normalization / cleaning

Normalize each channel with z-score or min-max scaling.

Clip outliers and filter obvious drift or sensor noise.

Keep units consistent so channels are comparable for learning.

Goal: stable, aligned multivariate time series $\mathbf{X} \in \mathbf{R}^{T \times d}$.

Unimodal Representation - Tables

Caption { Singapore Cup

Attribute {

Year	Champions	Runners-up
1996	Geylang United	Singapore Armed Forces
1997	Singapore Armed Forces	Woodlands Wellington
1998	Tanjong Pagar United	Sembawang Rangers

Cell {

Text { Singapore Armed forces was the champion of Singapore Cup in 1997.

Key issues in tabular data

Numeric columns: scale or normalize values.

Categorical columns: encode or learn embeddings.

Missing values: impute, flag, or mask consistently.

Tables mix headers, rows, cells, and heterogeneous feature types.

Prediction starts with clean row representations.

Tabular embeddings

Represent each row by combining embedded categorical features with normalized numeric features.

Categorical columns

Learn a small embedding for each category value.

Examples: country, device type, product ID, user segment.

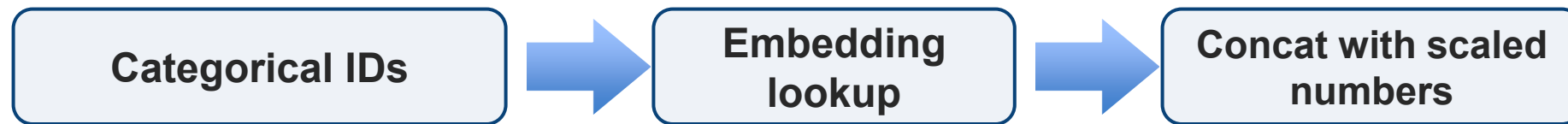
Lookup table: store one d-dimensional vector for each category ID.

Numeric columns

Keep values as **normalized scalars** after z-score or min-max scaling.

Optionally **bucketize** or pass through a small MLP.

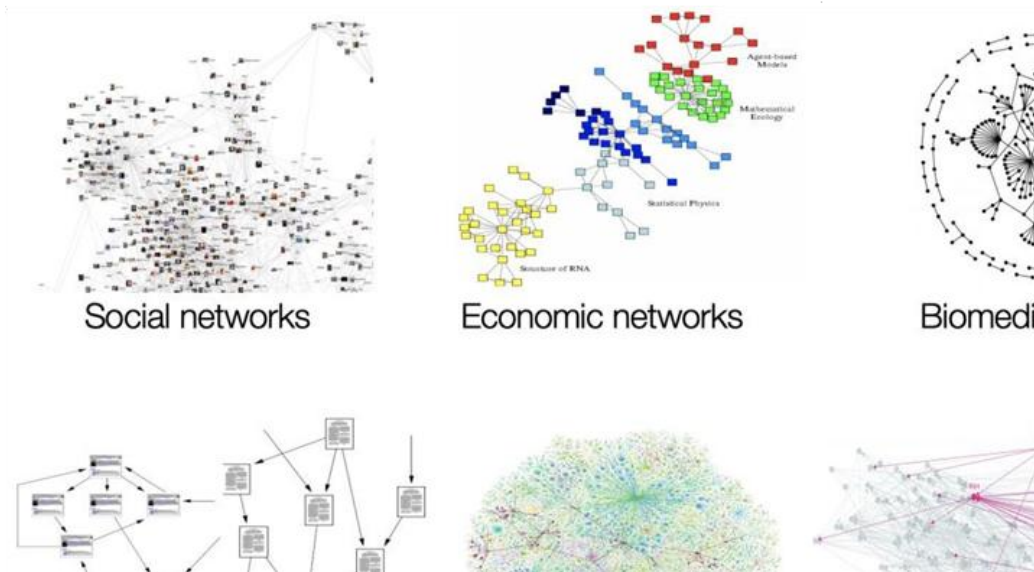
Handle missing values **consistently** so training sees stable inputs.



Row vector = [emb(country) ; emb(device) ; price_norm ; age_norm]

Unimodal Representation - Graphs

Structure + attributes in graphs



Different domains reuse the same graph abstraction:
nodes are entities and edges are relations.

1. Adjacency matrix

$A \in \{0,1\}^{(N \times N)}$

$A_{ij} = 1$ when nodes i and j are connected.

2. Edge list + node features

$E = [(u,v), \dots]$ and $X \in \mathbb{R}^{(N \times d)}$

Compact for sparse graphs; X stores d attributes per node.

3. What the model sees

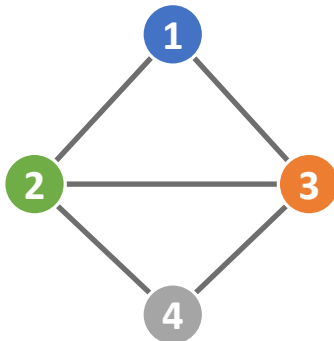
Topology plus node/edge attributes drive node, link, and graph tasks.

Graphs are not just nodes and edges - they also need explicit numeric representations.

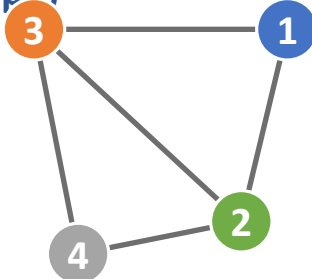
Graph invariance

Permutation of node IDs should not change the meaning.

Ordering 1



Ordering 2 (same graph)



$$A' = P A P^T$$

The node order changes, but the underlying graph does not.

Message passing intuition

$h_v^{(k+1)} = \text{Update}(h_v^{(k)}, \text{Aggregate}(\{h_u^{(k)} : u \in N(v)\}))$
Each node mixes its own state with information from neighbors.

Permutation-invariant aggregation

Common choices: sum, mean, or max over neighbor messages.

Good graph models are invariant or equivariant to node reordering.

Unimodal Representation - Sets



Sets



Point clouds



Set anomaly
detection
Set expansion
Set completion
Point cloud
classification
Point cloud
generation

Key property:
permutation invariance

A set has no canonical order.
Reordering elements should not
change the prediction.

Useful for point clouds, set completion, and anomaly detection.

DeepSets formula

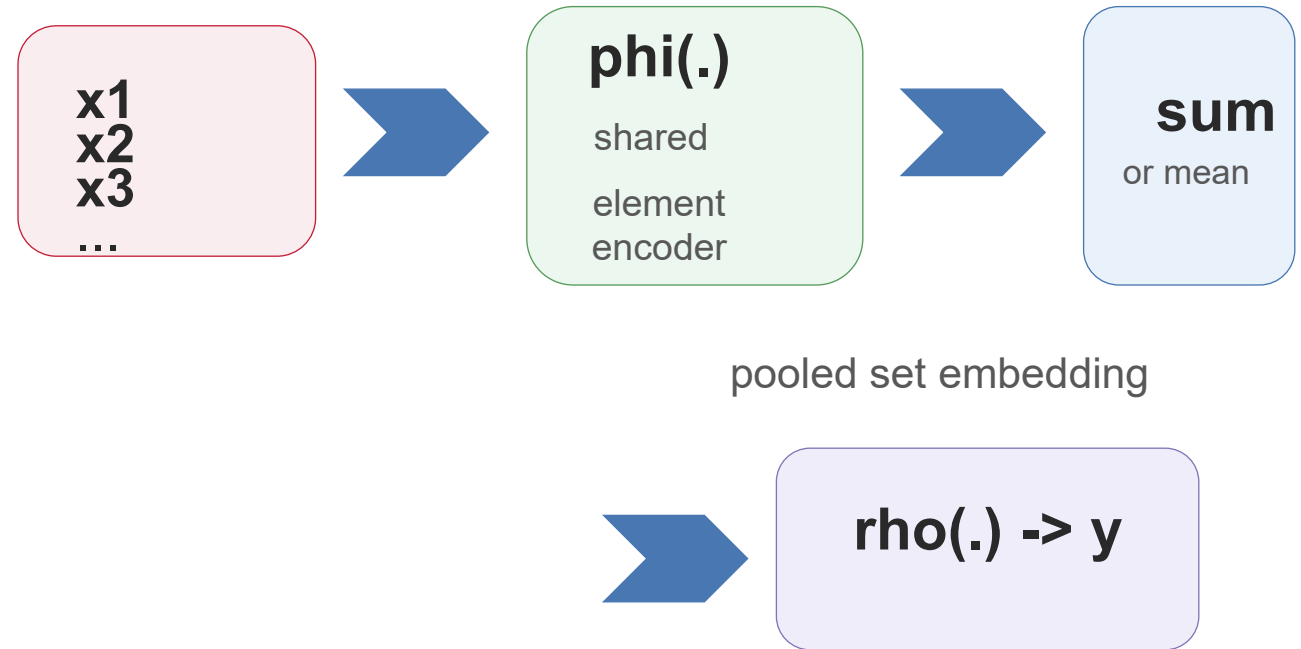
Permutation-invariant set

Set embedding = $\rho(\sum_i \phi(x_i))$

$\phi(x_i)$: shared element encoder
sum / mean: order-invariant pooling
 $\rho(\cdot)$: task-specific set predictor

Why sum or mean pooling works:
commutative aggregation gives the same
output for any permutation of the elements.

How the model processes a set



Dimensions of Heterogeneity

How modalities differ and why that changes modeling choices.

Different structure, noise, alignment, and invariances imply different preprocessing and architectures.

Image
2D grid

Text
sequence

Audio
time-
frequency

Graph / Set
relational /
unordered

Heterogeneous Modalities

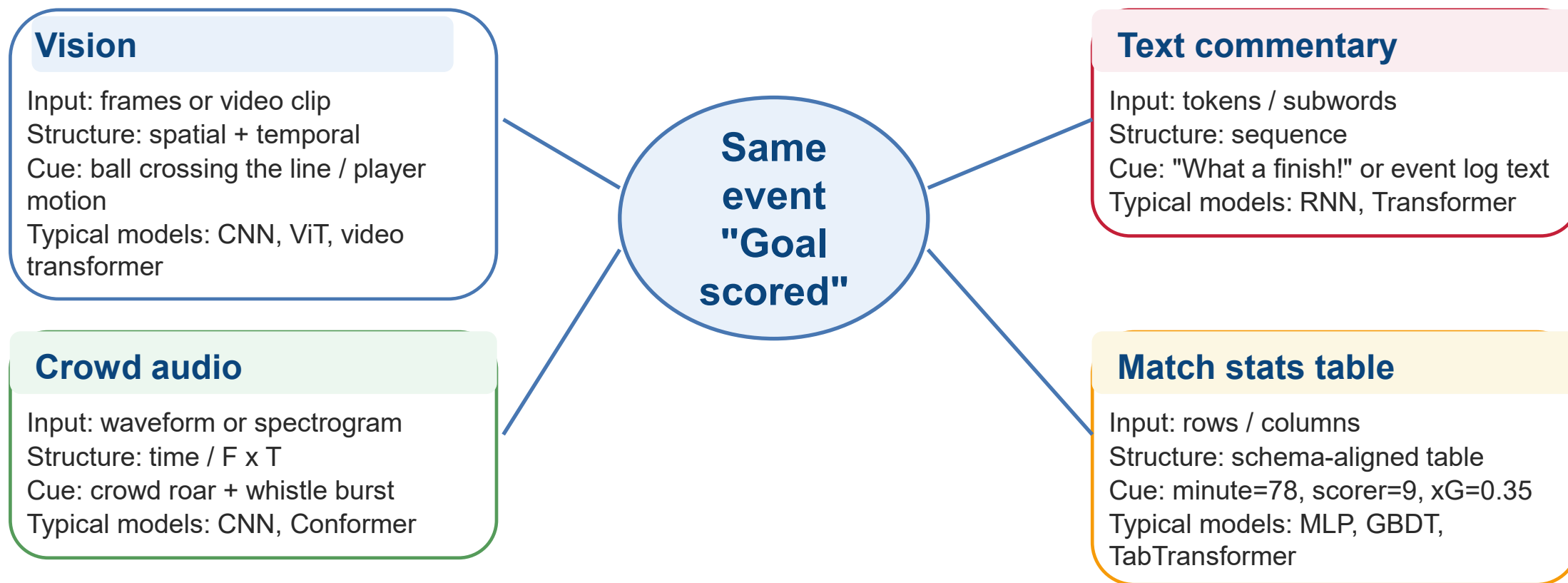
Concrete comparison: the same learning problem looks different depending on the modality.

Property	Image	Text	Audio	Table	Graph	Set
Element type	pixels / patches	tokens / subwords	samples / frames	numeric + categorical	nodes + edges	unordered elements
Structure	2D spatial grid	1D sequence	time or F x T	rows / columns	relational topology	no fixed order
Noise / missingness	blur, occlusion	typos, OOV	noise, reverb	missing cells	missing edges	outliers, missing items
Alignment	pixel grid	token order	time aligned	schema aligned	edge connectivity	set membership only
Typical invariances	translation scale (approx)	local context sensitivity	small time shifts	row permutation (often)	node permutation	permutation invariance
Typical models	CNN, ViT	RNN, Transformer	CNN, Conformer	MLP, GBDT, TabTransform	GNN	DeepSets, Set Transformer

Takeaway: representation choice, preprocessing, *and* inductive bias should follow the modality profile.

One worked example: same event across modalities

Example event: a soccer goal can be observed as vision, text, audio, and statistics.



Dimensions of Heterogeneity

- 1 **Element representations:**
Discrete, continuous, granularity
Example: text uses tokens; audio uses real-valued amplitudes.
- 2 **Element distributions:**
Density, frequency
Example: rare named entities vs dense image pixels.
- 3 **Structure:**
Temporal, spatial, latent, explicit
Example: sentences are sequential; graphs are relational.
- 4 **Information:**
Abstraction, entropy
Example: captions are high-level; raw images keep low-level detail.
- 5 **Noise:**
Uncertainty, noise, missing data
Example: reverberation in audio; empty cells in tables.
- 6 **Relevance:**
Task and context dependence
Example: color matters for fruit classification, but not for sentiment.

Why these dimensions matter

They determine what preprocessing is needed.

They suggest what invariances the model should encode.

They explain why one architecture rarely fits every modality.

They help compare modalities before designing a multimodal system.

Modality Profile

1. Element representation

Are the basic units discrete or continuous, and how fine-grained are they?

2. Element distribution

Are observations sparse or dense, frequent or rare?

3. Structure

Is organization spatial, temporal, relational, latent, or explicit?

4. Information

Does the modality carry raw low-level detail or abstract semantic content?

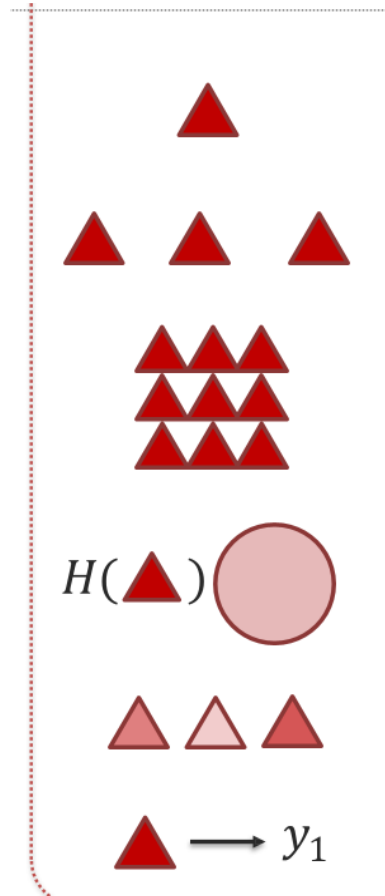
5. Noise

How much uncertainty, corruption, or missing data is typical?

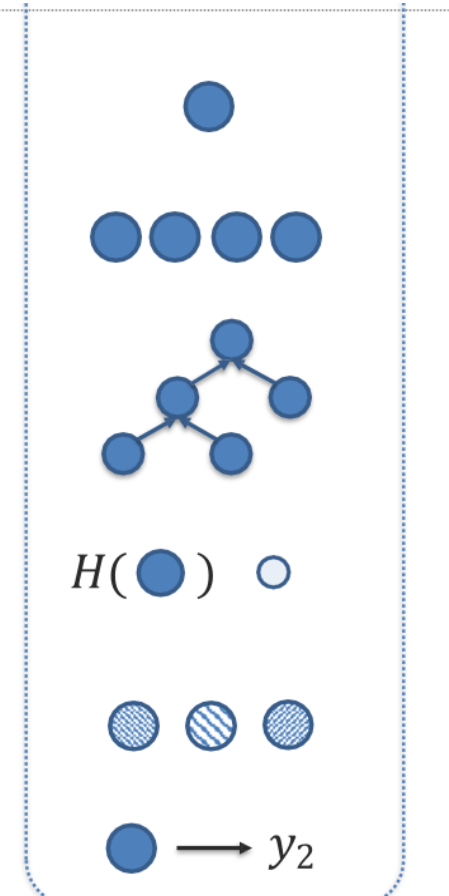
6. Relevance

Which information matters for the downstream task?

Modality A



Modality B



A modality profile compares modalities along the same six axes so the design trade-offs are explicit.

Modality profiles

Concrete examples for image, text, and audio modalities.

Image

Elements: continuous pixels / patches
Distribution: dense over a 2D grid
Structure: explicit spatial layout
Information: rich low-level detail
Noise: blur, lighting, occlusion
Relevance: strongly task-dependent

Text

Elements: discrete tokens / subwords
Distribution: sparse and Zipfian
Structure: explicit sequence order
Information: abstract semantic content
Noise: ambiguity, typos, OOV terms
Relevance: syntax and context matter

Audio

Elements: continuous samples / frames
Distribution: dense over time
Structure: explicit temporal order
Information: acoustic + semantic cues
Noise: background, reverb, channel effects
Relevance: task decides the useful time scale

Modality Profile - architecture implications

Profile summary: modality properties should drive architecture choice and inductive bias.

Profile clue	Useful inductive bias	Common model families
Local 2D neighborhoods and translation matters	Locality + weight sharing	CNN, ConvNeXt, ViT with patches
Long sequential context and order matters	Temporal / causal context modeling	RNN, Transformer, Conformer
Elements are unordered	Permutation-invariant pooling	DeepSets, Set Transformer
Relational topology carries meaning	Message passing across edges	GNN, graph transformer
Mixed schema + missing values	Column-wise encoding + robust preprocessing	GBDT, MLP, TabTransformer

Takeaway: pick the model family that matches the modality structure before adding task-specific detail.

Image Representations

Agenda

Object-based representation

Grid / pixels and hand-crafted descriptors

Learned CNN features

How Would You Describe This Image?



88
88
85
38
20
22
24
21
23
82
80
79
35
25
26
28
22
22
84
78
80
⋮

Answer example

Indoor diner scene: people, cups, table, conversation context

Three Levels of Description



Low-level description

Edges, colors, corners, local gradients; close to raw pixels.

Mid-level description

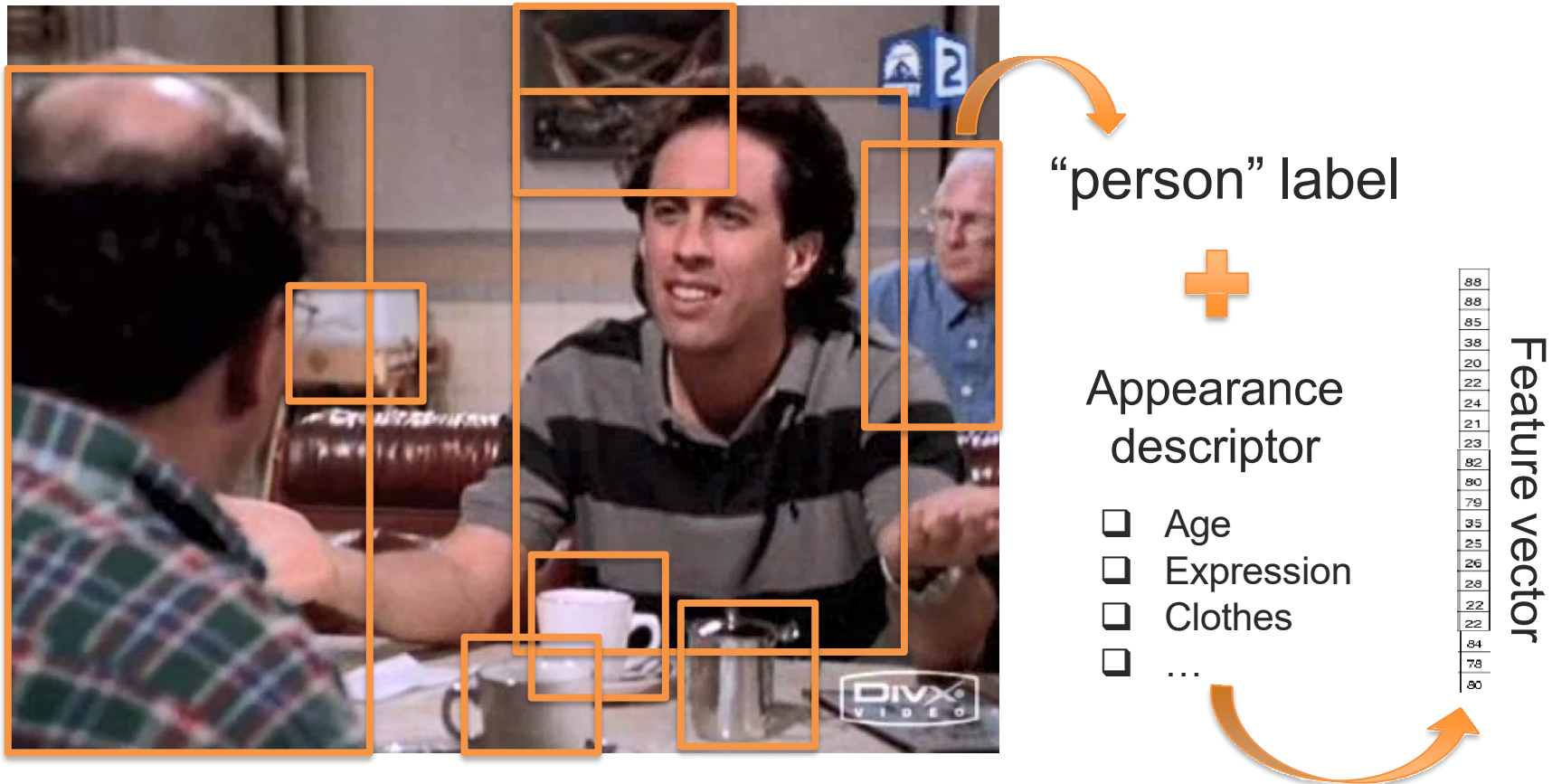
Parts, textures, repeated patterns, object fragments.

High-level description

Objects, relations, scene context, intent or interaction.

A good representation should support all three levels - from local evidence to semantic meaning.

Object-Based Visual Representation



How obtained / limitation

Detector proposes objects; attribute classifier predicts age, clothes, expression, etc.

Limitation: if detection is wrong or incomplete, the representation degrades.

When Object-Based Representations Help

Helps

VQA: questions about who / what / where

Retrieval: finding images with specific objects or attributes

Captioning and scene graphs: explicit entities and relations

Interpretability: easier to inspect than raw pixel features

Hurts / limitations

Fine-grained textures or materials may be missed

Errors in detection propagate downstream

Small / occluded objects may disappear

Spatial layout and context are only partially captured

Object-based features are useful when named entities matter, but they are not a complete description of an image.

Object Descriptors



How to represent and detect an object?

Many approaches over the years....

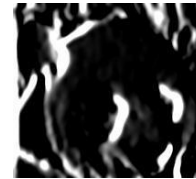
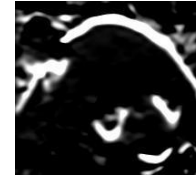
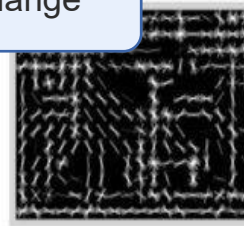


Image gradient

local intensity change



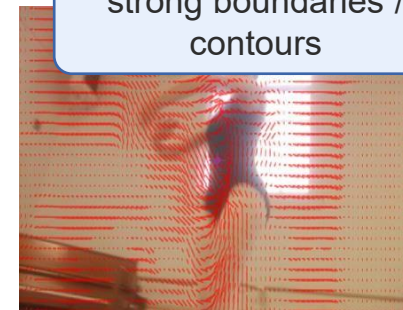
Histograms of Oriented Gradients

shape from gradient orientation histograms



Edge detection

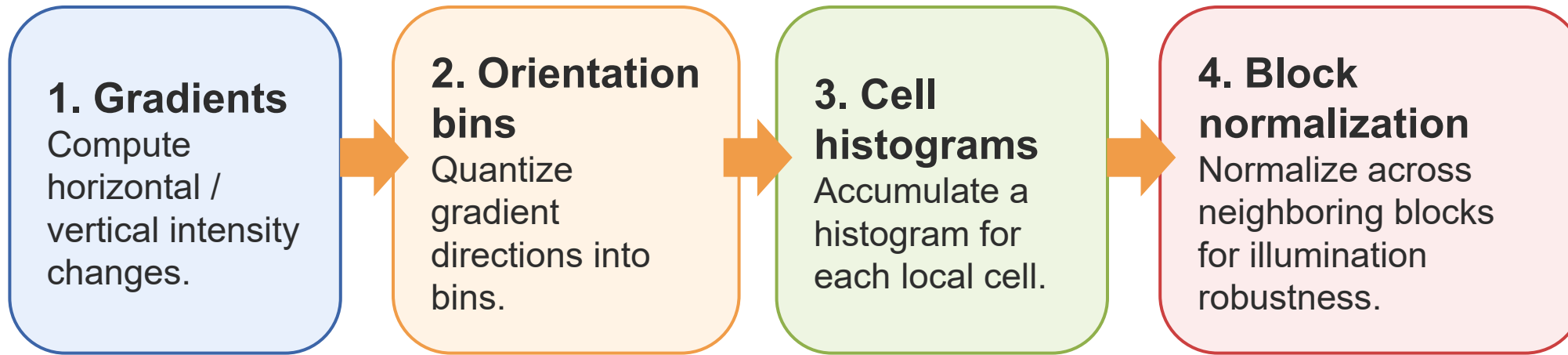
strong boundaries / contours



Optical Flow

motion between frames

HOG Mini-walkthrough



What HOG captures

HOG summarizes local shape by counting oriented gradients, so it is stronger than raw pixels but still hand-crafted.

Object Descriptors



How to represent and detect an object?

Many approaches over the years....

Horizontal
and vertical
gradients

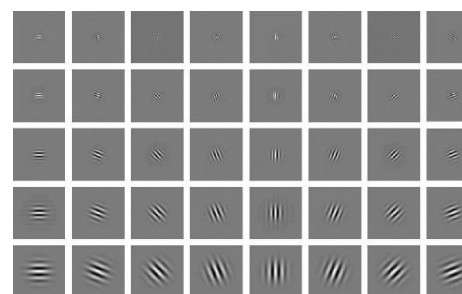


Oriented
gradients

Haar Wavelets



Templates tested
on the image
(i.e., *convolution
kernels*)



Gabor filters



Inspired by
visual cortex

Gabor filters respond to specific orientations and spatial frequencies - useful for textures and edge-like patterns.

Gabor Filters

Biologically-Inspired Feature Detectors

How a Gabor Filter is Constructed



Key Properties



Inspired by V1 Visual Cortex

Neurons in area V1 respond to specific orientations & spatial frequencies — Hubel & Wiesel, Nobel Prize 1981



Orientation × Frequency Selectivity

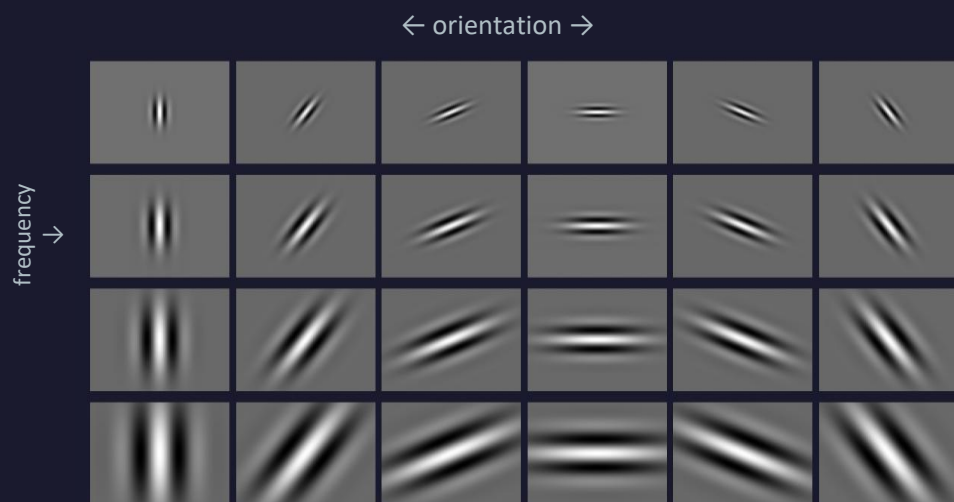
Each filter responds to one orientation and one spatial frequency — together they tile the full feature space



Texture, Fingerprint & Iris Recognition

Powerful for capturing edge-like patterns and periodic textures in classical computer vision

Gabor Filter Bank

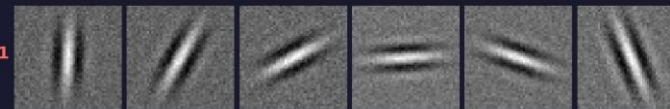


Deep Learning Connection

Hand-designed



CNN Layer 1



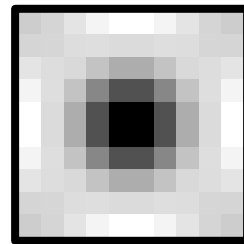
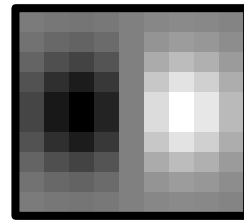
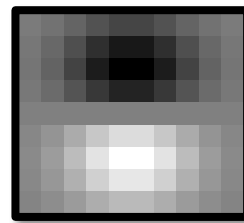
CNN Layer 1 filters independently rediscover Gabor-like patterns — same operation, learned weights

Key Insight: Gabor filters are fixed convolution kernels. CNNs learn similar kernels from data — then go further, composing them into hierarchical features that hand-crafted methods could never scale to.

Convolution Kernels

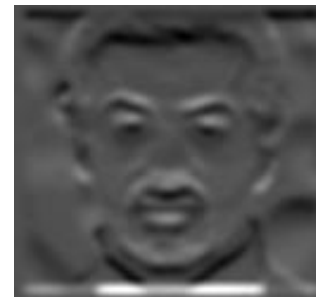


Example: 3x3 kernel, stride = 1, padding = 0



Convolution
kernels

=



Response maps

Convolution Hyperparameters



Kernel size (K)

Spatial extent of the filter, e.g. 3x3 or 5x5.

Stride (S)

How far the kernel moves at each step.

Padding (P)

Zeros added around the border to control output size.

Dilation (D)

Spacing between kernel elements; grows receptive field.

Output size

$H_{out} = \text{floor}((H + 2P - D(K-1) - 1) / S + 1)$, and similarly for W_{out} .

Object Descriptors

Why not only objects?

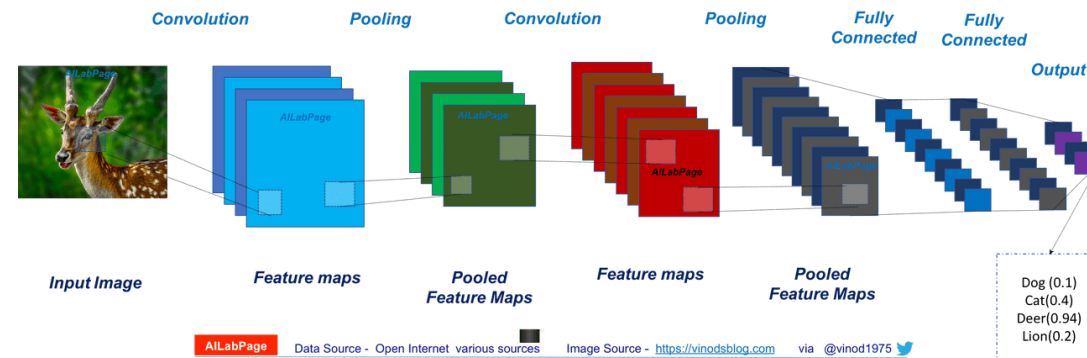
Spatial layout matters
Context matters



How to represent and detect an object?

Many approaches over the years....

Convolutional Neural Network (CNN)



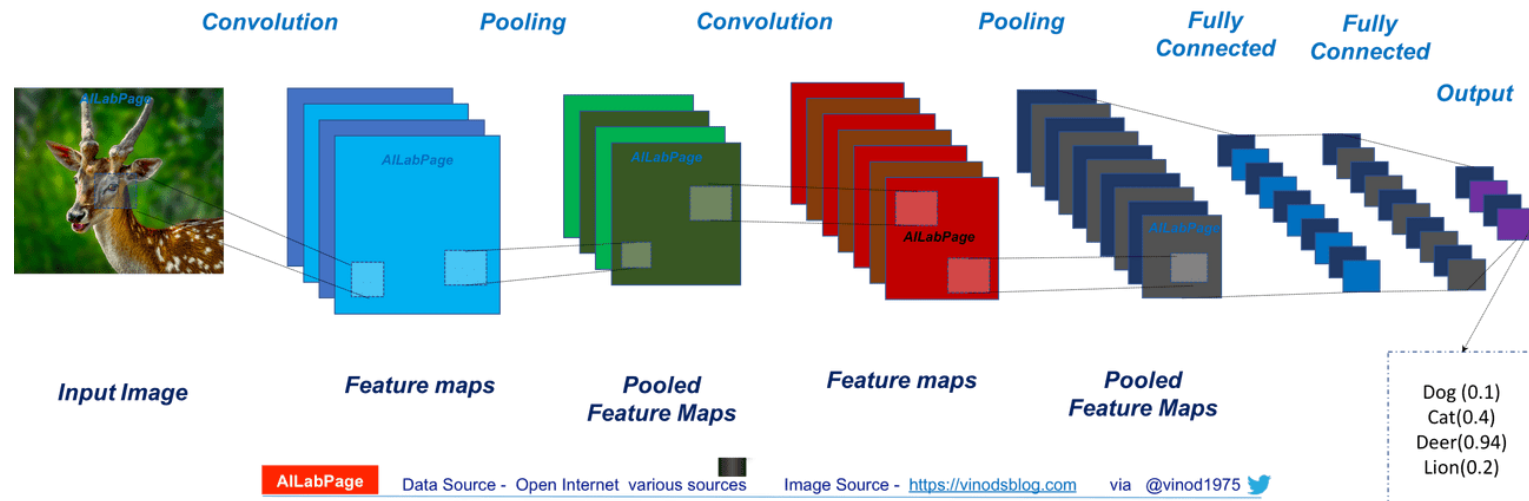
More details about CNNs is coming...
... and we will also talk about visual transformers in coming weeks...

And images are more than a list of objects!

Bridge: Why Learned Features?

Why move beyond hand-crafted features?

Hand-crafted pipelines saturate: they capture only what we design. End-to-end learning adapts features to the task and data. Hierarchical features capture edges -> parts -> objects -> context. Pretraining and transfer learning make learned features reusable.



This motivates CNNs: learn the filters instead of fixing them by hand.

Convolutional Neural Networks

CNN = learned kernels + nonlinearities + pooling/striding → hierarchical features.



Why using Convolutional Neural Networks?

Goal: building more abstract, hierarchical visual representations

Key advantages:

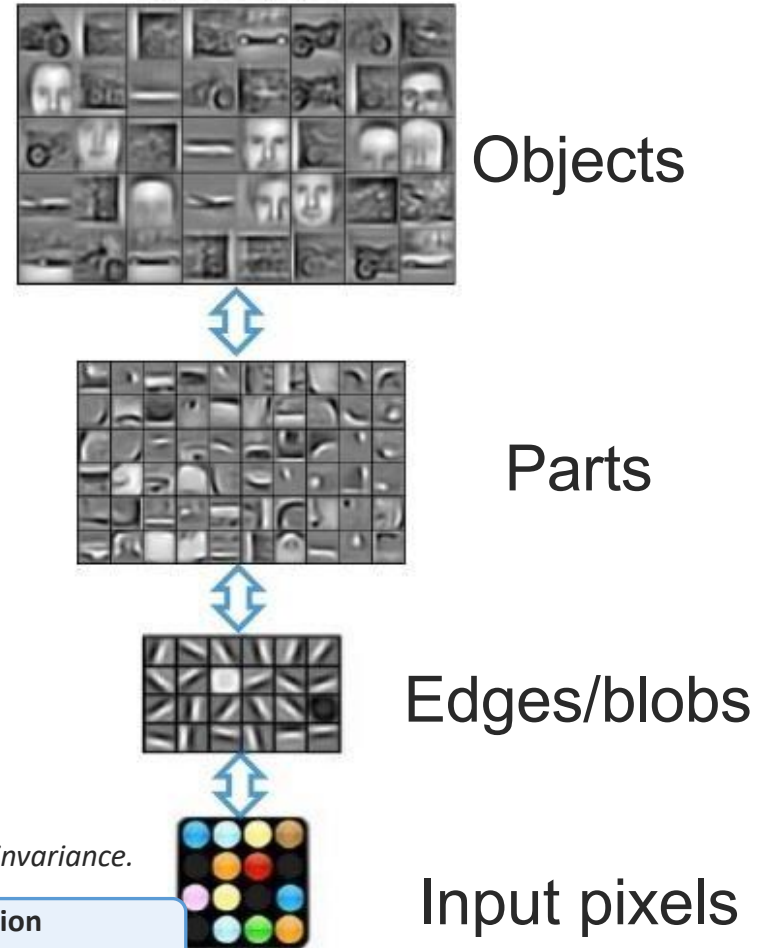
- 1) Inspired from visual cortex
- 2) Encourages visual abstraction
- 3) Exploits *translation invariance*
- 4) Kernels/templates are learned
- 5) Fewer parameters than MLP

Convolution is translation-equivariant; pooling/global pooling helps create invariance.

local receptive
fields

parameter
sharing

translation
equivariance



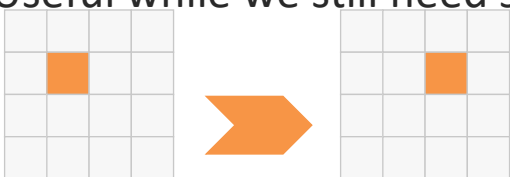
Equivariance vs Invariance

Convolution preserves spatial structure early; pooling and global aggregation create more location-robust representations.

Equivariance

If the input shifts, the feature map shifts in the same way.

Useful while we still need spatial localization.



Input shifted right → response map also shifts right

Invariance

After max/avg pooling or global average pooling, the representation or class score can stay similar.



Pooling/global pooling → more location-robust summary

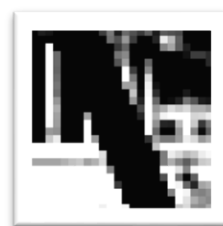
Convolution in 2D – Example

Single output pixel:
 $\text{dot}(\text{local patch}, \text{kernel})$



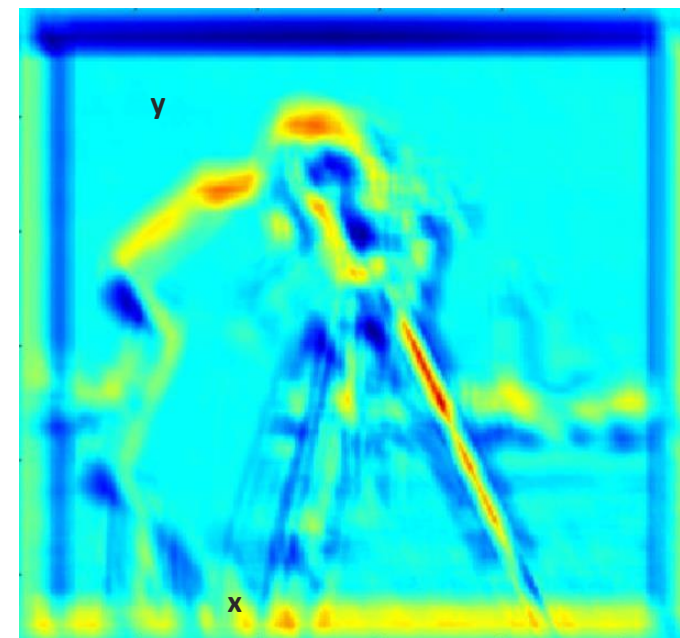
Input image

*



Convolution
kernel

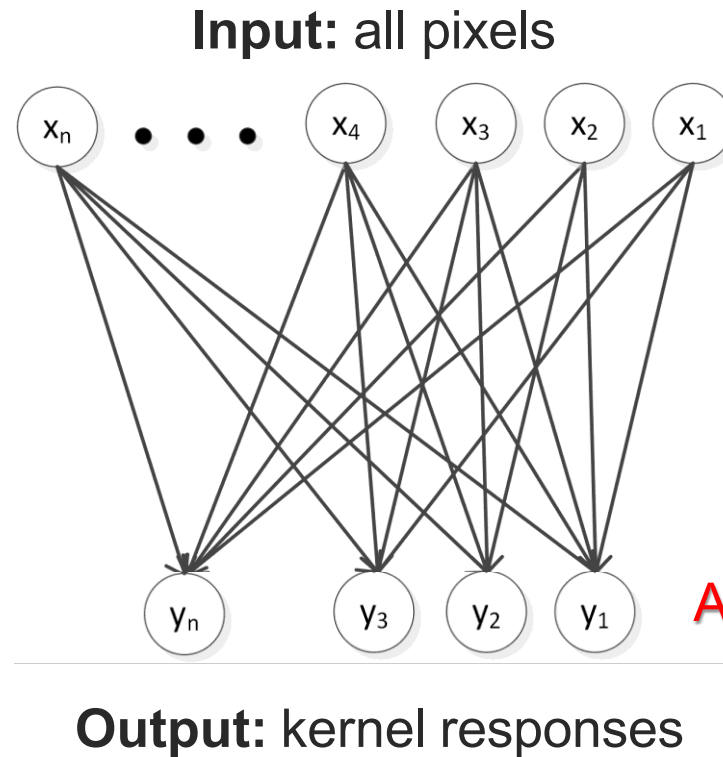
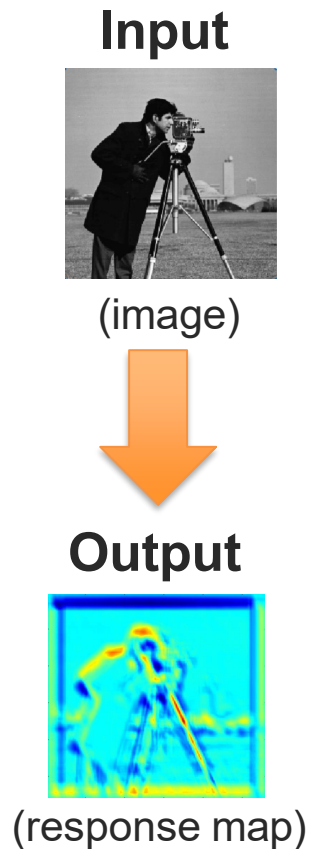
=



Response map

$$y[i, j] = \sum_u \sum_v K[u, v] \cdot X[i+u, j+v]$$

Convolution as a Fully-Connected Network



General parameter counts
FC: $(H \times W \times C) \times K + K$
Conv: $(k_h \times k_w \times C_{in}) \times F + F$

Not efficient!

200 × 200 image
requires
40,000 × n parameters
(where n is size of kernel)

**And it may learn different kernels
for different pixel positions**

➔ Not translation invariant

Convolutional Neural Layer

Input

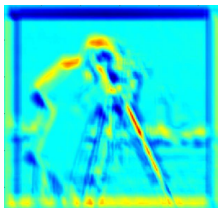


(image)



Weighted sum
 Wx

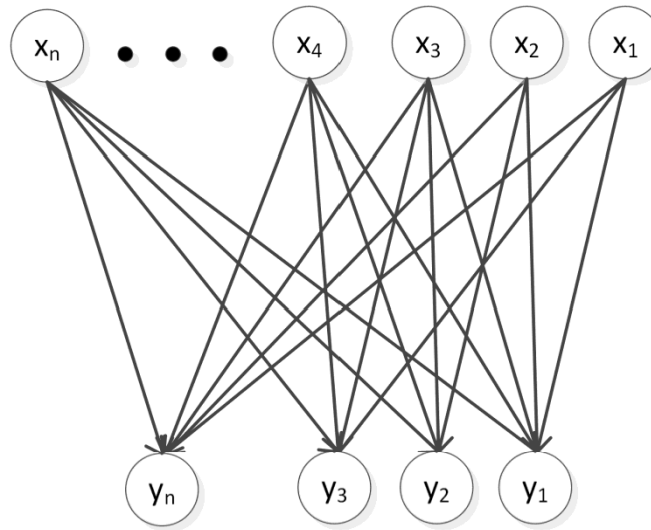
Output



(response map)

$$y = Wx$$

Input: all pixels



Output: kernel responses

Stride notation

$s = 1$ here

Shared weights are reused at
each location

**Example with
1D kernel:**

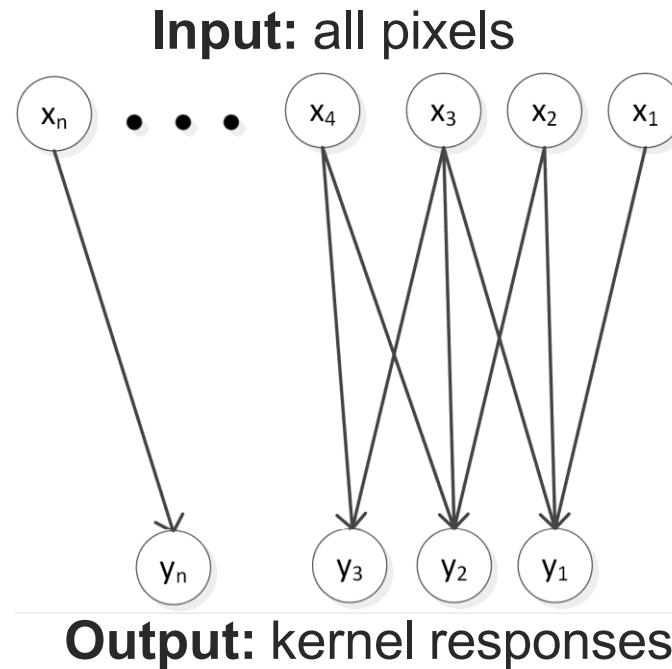
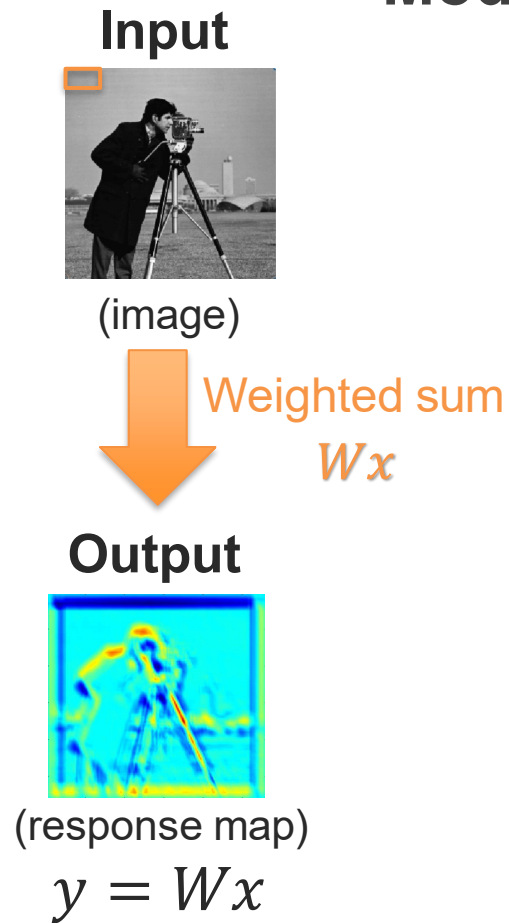
w_1	w_2	w_3
-------	-------	-------



Convolution
kernel

Convolutional Neural Layer

Modification 1: Sliding window – Only apply the kernel to a small region



Example with 1D kernel:

w_1	w_2	w_3
-------	-------	-------

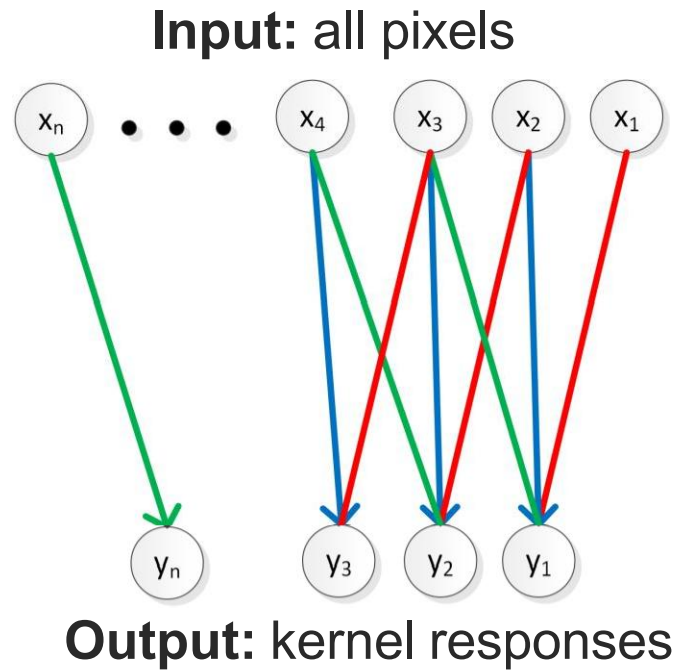
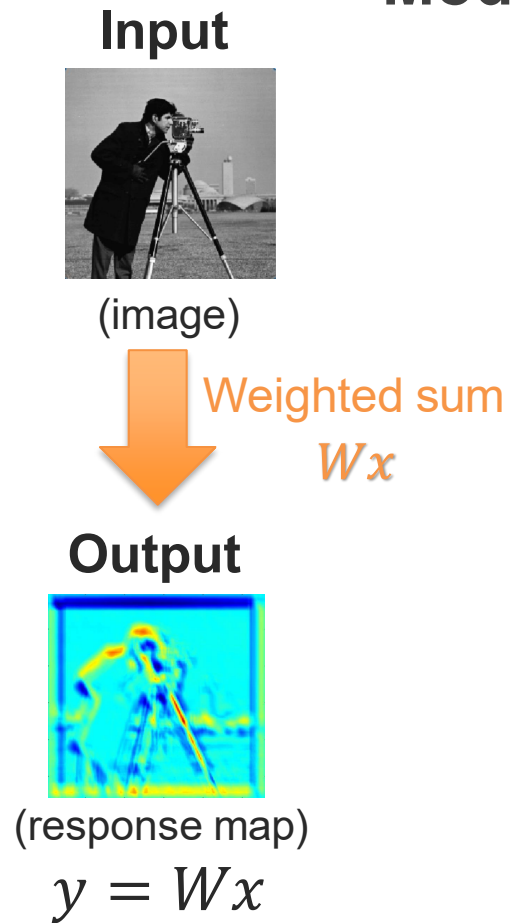


Padding / stride variants

- padding $p = 0 \rightarrow$ output shrinks
- stride $s = 1 \rightarrow$ dense sampling
- stride $s = 2 \rightarrow$ downsampling

Convolutional Neural Layer

Modification 2: Same kernel applied to all sliding windows



Example with
1D kernel:

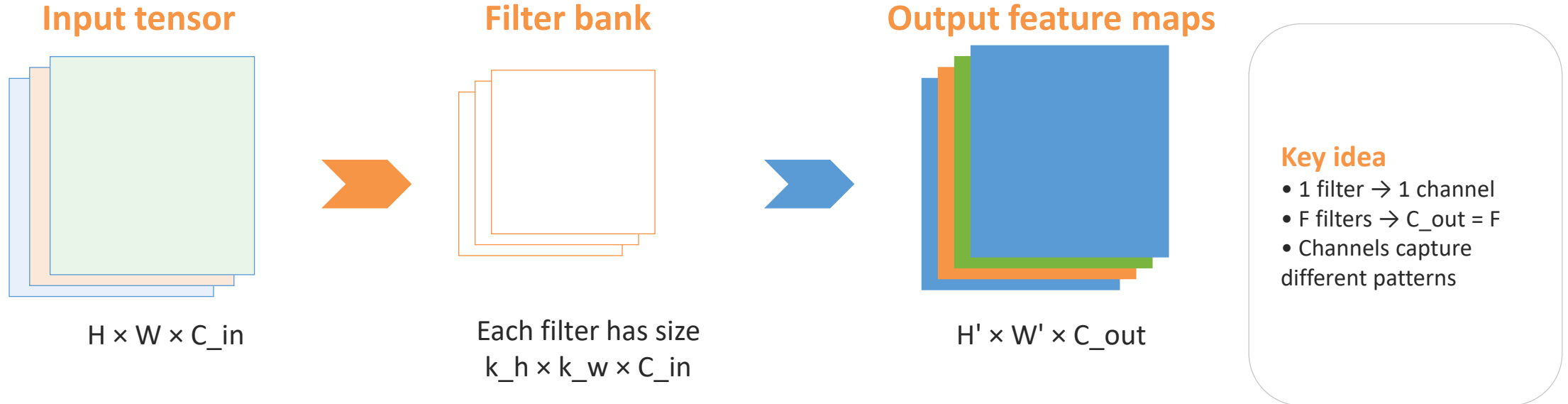


Multiple filters → multiple
channels

Each filter produces one response map.
 F filters $\Rightarrow C_{out} = F$ output channels.

Multi-channel Convolution

One filter spans all input channels; each filter produces one output channel. Many filters produce a feature tensor.



Convolutional Neural Layer

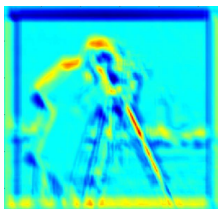
Input



(image)



Output



(response map)

$$y = Wx$$

Modification 2: Same kernel applied to all sliding windows

$$W = \begin{pmatrix} w_1 & w_2 & w_3 & & 0 & 0 & 0 \\ 0 & w_1 & w_2 & \dots & 0 & 0 & 0 \\ 0 & 0 & w_1 & & 0 & 0 & 0 \\ & \vdots & & \ddots & \vdots & & \\ 0 & 0 & 0 & & w_3 & 0 & 0 \\ 0 & 0 & 0 & \dots & w_2 & w_3 & 0 \\ 0 & 0 & 0 & & w_1 & w_2 & w_3 \end{pmatrix}$$

Convolution = a linear map with a sparse, weight-shared matrix.
This is why conv is a constrained linear layer.

Example with
1D kernel:



➡ Can be implemented efficiently on GPUs

➡ W will be 3D: 3rd dimension allows for multiple kernels

Convolutional Neural Network

Multiple convolutional layers

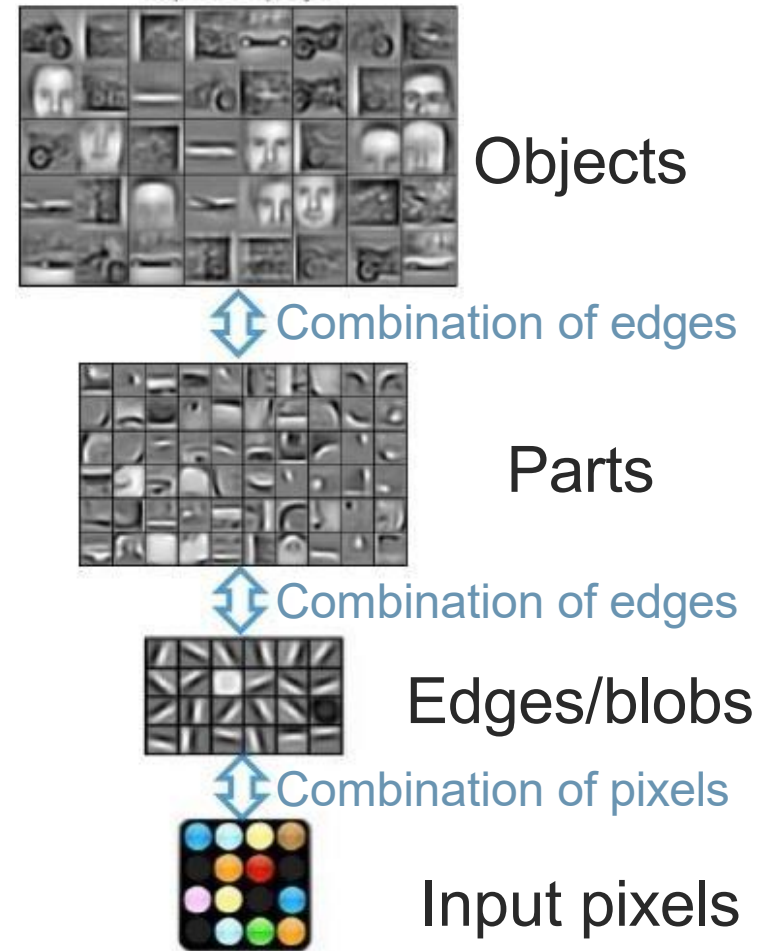
→ Allows the network to learn combinations of sub-parts, to increase complexity

Example stack

[Conv 3x3 → BN → ReLU] × 2 → MaxPool

but how to encourage abstraction and summarization?

Answer: Pooling layers



Training CNNs in Practice

A strong pipeline matters as much as the architecture.

Data augmentation

- crop / resize
- flip
- color jitter
- mixup / cutmix

Normalization

- BatchNorm
- stable activations
- use ReLU / GELU

Optimization

- SGD or AdamW
- weight decay
- learning-rate schedule

Transfer learning

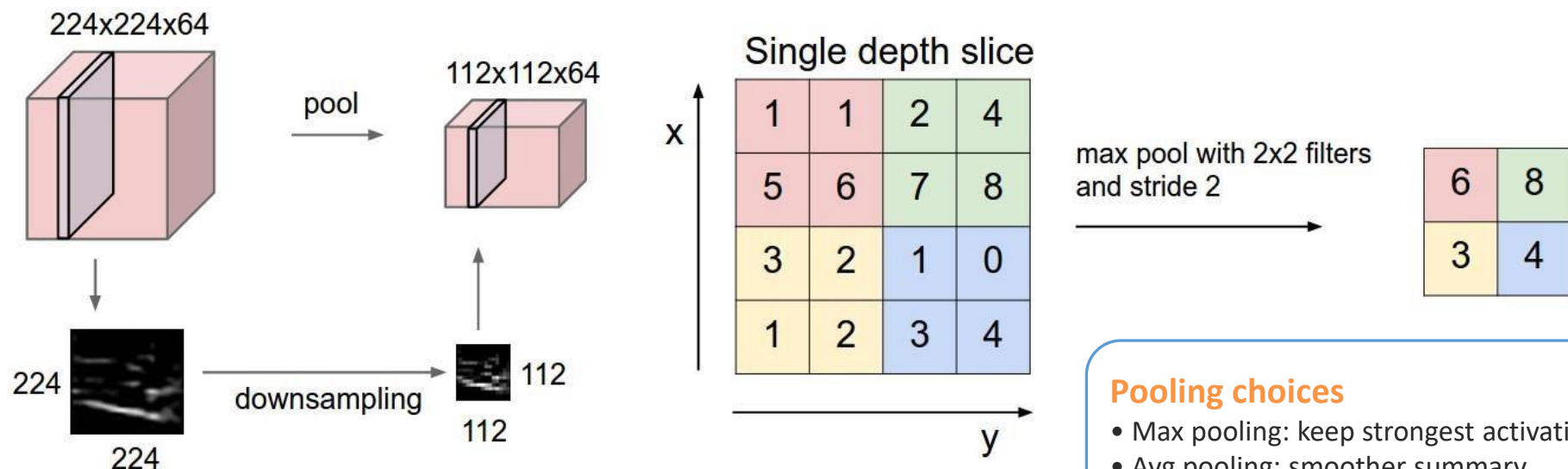
- pretrained backbone
- fine-tune on your task

Practical recipe

- Monitor validation accuracy / loss, not just training loss.
- Use early stopping or checkpoint selection.
- Smaller datasets benefit a lot from pretrained features.

Pooling Layer

Response map subsampling:
Allows summarization of the responses

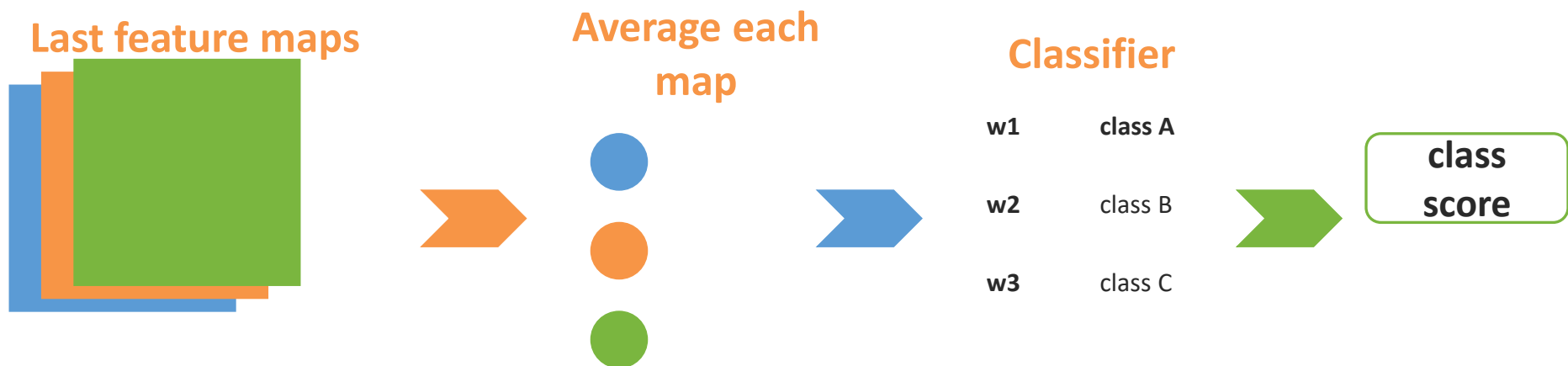


Pooling choices

- Max pooling: keep strongest activation
- Avg pooling: smoother summary
- Strided conv: learned downsampling

Global Average Pooling (GAP)

GAP averages each feature map into one scalar, reducing parameters and making class activation mapping (CAM) easy.



Why it is useful

- Fewer parameters than a big fully-connected head.
- GAP + linear classifier makes CAM easy: class score is a weighted sum of feature maps.
- Often used at the end of modern image classifiers.

Common CNN Architectures

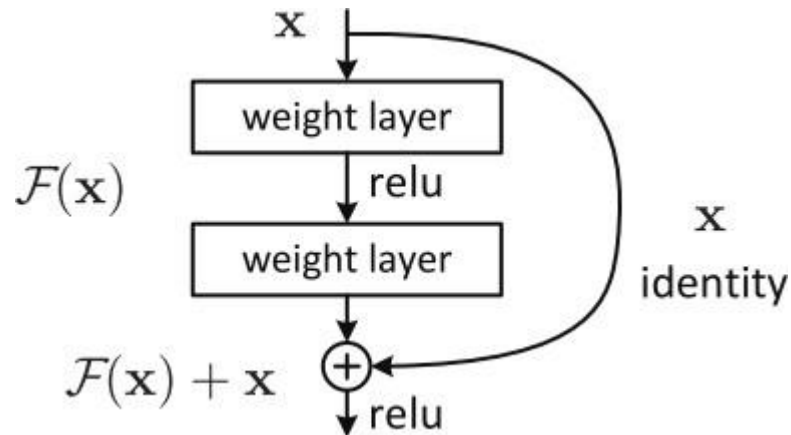
Mini-timeline from classic to modern ConvNets

Model	Year	Main idea	Why it mattered
LeNet	1998	small convnet	Showed end-to-end gradient-based learning for image recognition.
AlexNet	2012	ReLU + dropout + GPUs	Restarted deep vision by winning ImageNet decisively.
VGG	2014	very deep stacks of 3×3 convs	Simple, uniform design; strong transfer baseline.
Inception	2014	multi-scale branches	Captured different receptive fields efficiently.
ResNet	2015	skip / residual connections	Made very deep networks easier to optimize.
ConvNeXt	2022	modernized pure ConvNet	Showed convnets remain highly competitive in the transformer era.



Residual Networks (ResNet)

Adding residual connections

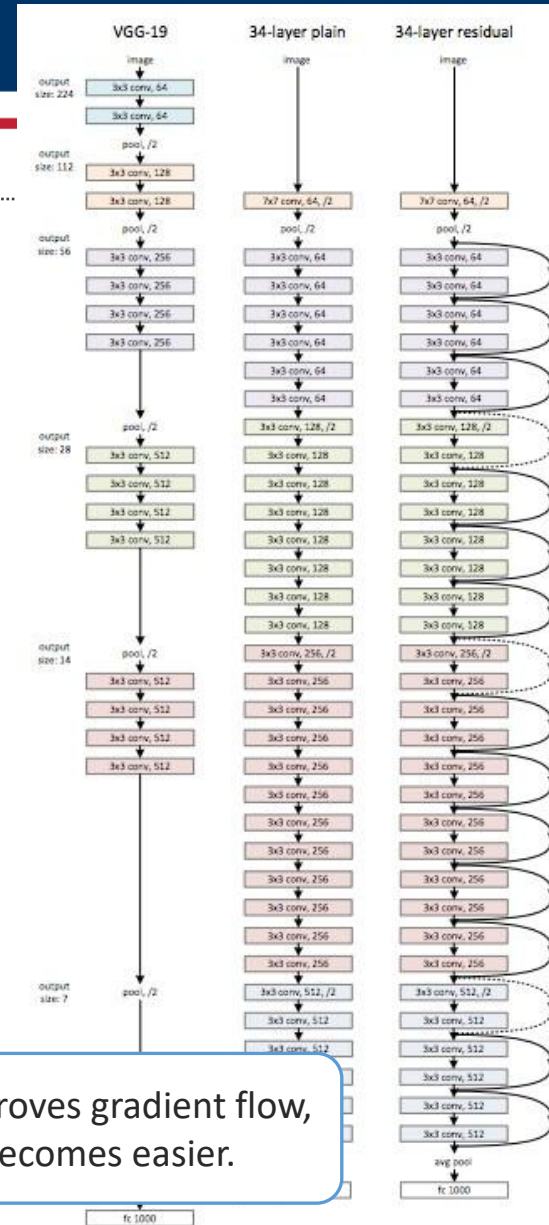


ResNet (He et al., 2015)

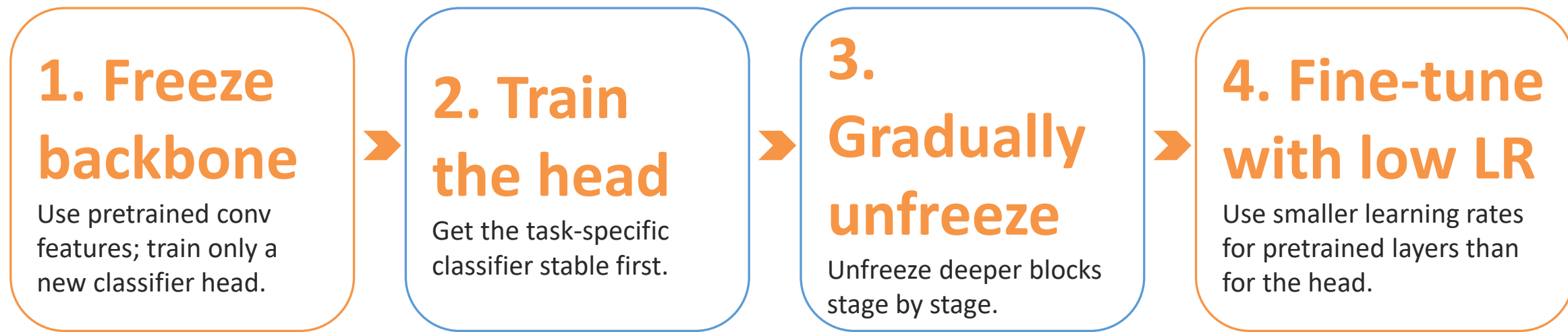
- Up to 152 layers!

Residual block:
 $y = F(x) + x$

Identity skip path improves gradient flow,
so optimization becomes easier.



A practical fine-tuning workflow for image tasks



Why it helps

- Works well when labeled data is limited.
- Faster convergence than training from scratch.
- Very common for detection, segmentation, and medical imaging.

Two-stage detection

First propose candidate regions, then classify and refine each region.

Object Detection (and Segmentation)



Input image



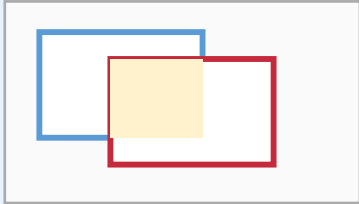
Detected Objects

One option: Sliding window

Why expensive? Many windows x many scales x many aspect ratios -> repeated detector evaluations.

Detection Basics

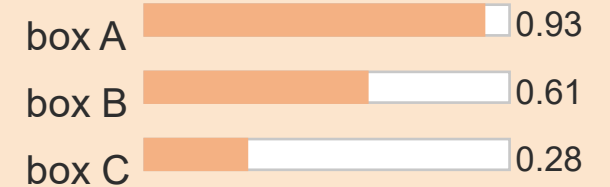
IoU



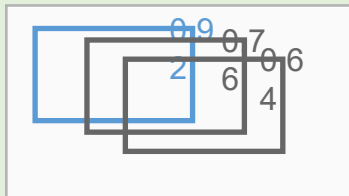
$\text{IoU} = \text{overlap} / \text{union}$
Higher IoU means the predicted box matches the ground truth better.

Confidence score

score = objectness
x class probability



Non-maximum suppression



Keep the highest-score box
Suppress nearby boxes with high overlap.

mAP



Area under the precision-recall curve
Mean AP averages this across classes (and often IoU thresholds).

Object Detection (and Segmentation)



A better option: Start by Identifying hundreds of region proposals and then apply our CNN object detector

Proposal methods: Selective Search -> learned Region Proposal Networks (RPNs)

Two-stage vs One-stage Detectors

Two-stage (Faster R-CNN)

proposals -> shared features -> RoI head ->
class + box

Often higher accuracy; strong on small /
crowded objects

More flexible detection heads and refinements

Usually slower at inference

One-stage (YOLO / SSD)

image -> dense predictions -> class + box in
one pass

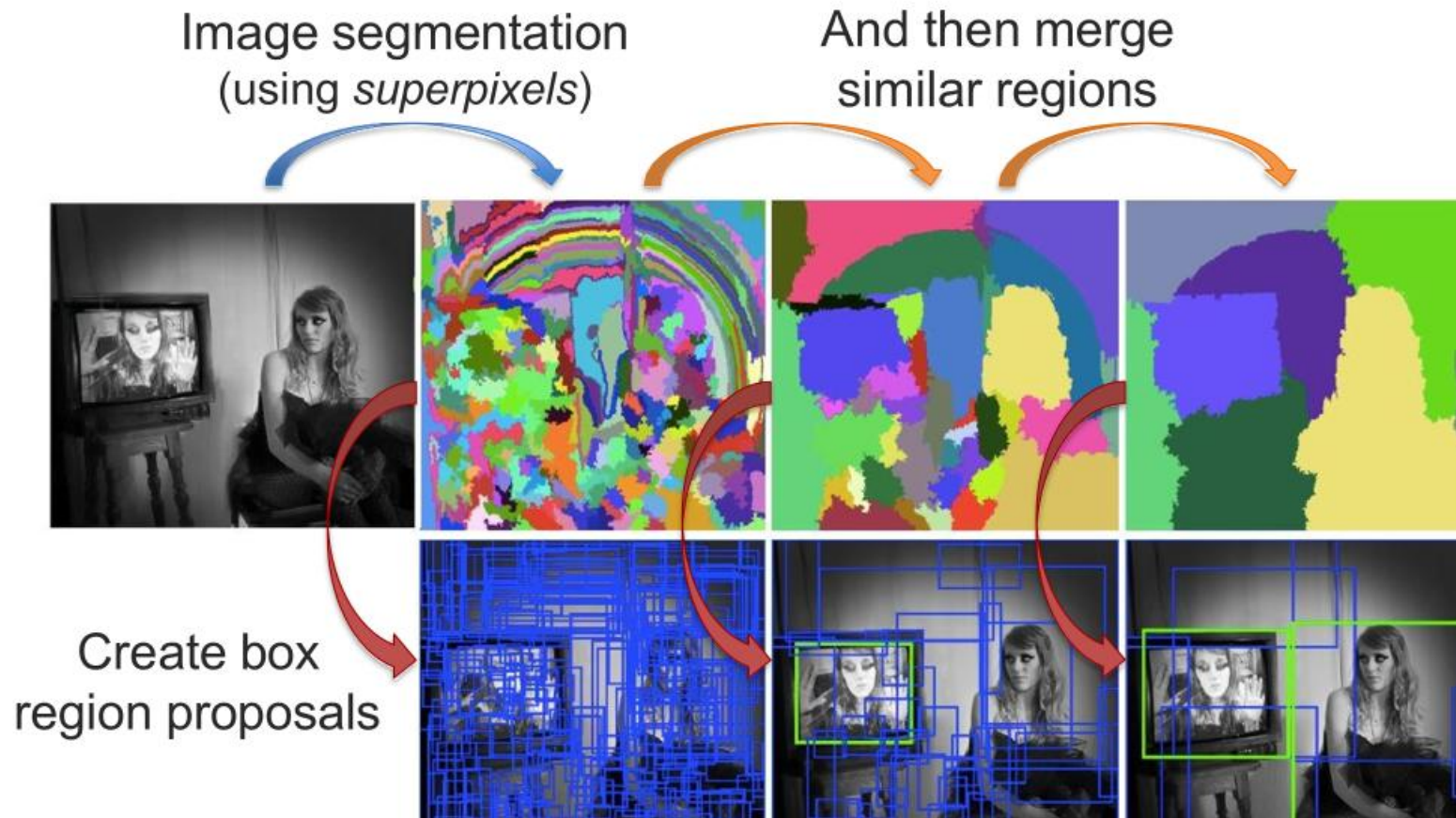
Faster and simpler deployment

Real-time use cases are common

May trade some accuracy for latency

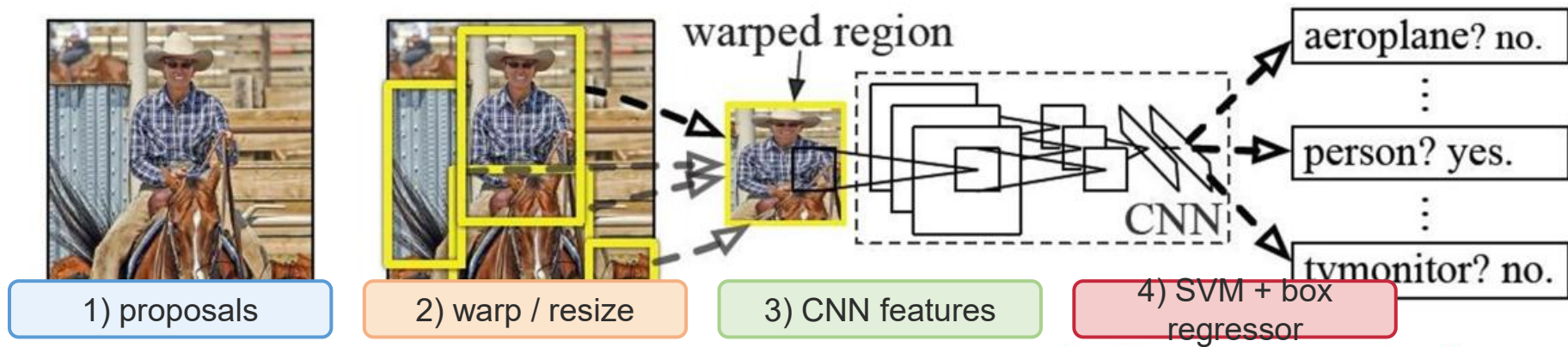
Tradeoff: choose the detector family based on latency, compute, and
accuracy needs.

Selective Search [Uijlings et al., IJCV 2013]



Why it was replaced: slow, hand-designed, and not learned jointly with the detector.

R-CNN [Girshick et al., CVPR 2014]



- Select ~2000 region proposals ➡ Time consuming!
- Warp each region
- Apply CNN to each region ➡ Time consuming!

Fast R-CNN: Applies CNN only once, and then extracts regions

Faster R-CNN: Region selection on the Conv5 response map

Slow because the CNN runs on each proposal crop.

Modern Detectors

Faster R-CNN

RPN generates proposals
RoI head predicts class + box
Strong accuracy, but slower

SSD / YOLO

Single-pass dense predictions
Good speed / latency tradeoff
Popular for real-time use

DETR

Transformer-based set prediction
End-to-end matching instead of hand-made proposals
Clean formulation; different training dynamics

All predict class + location, but they differ in how proposals are generated and refined.



Interpretability = what features or regions drive a model's decision.

Goal: connect internal feature maps to human-understandable evidence.

Visualizing the Last CNN Layer: t-SNE

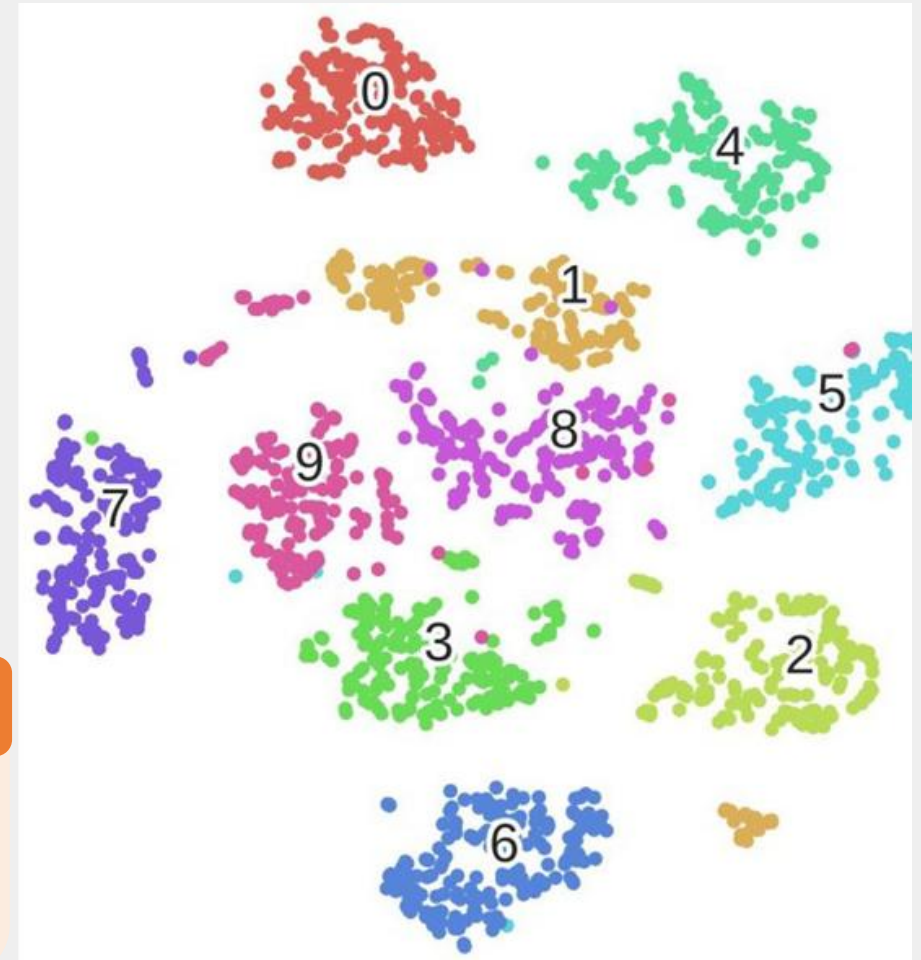
What it does

- Projects high-dimensional CNN feature codes into 2-D.
- Preserves local neighborhoods: nearby points tend to stay nearby.
- Useful for class clusters, overlap, and outliers.

**CNN features -> 2-D embedding ->
cluster visualization**

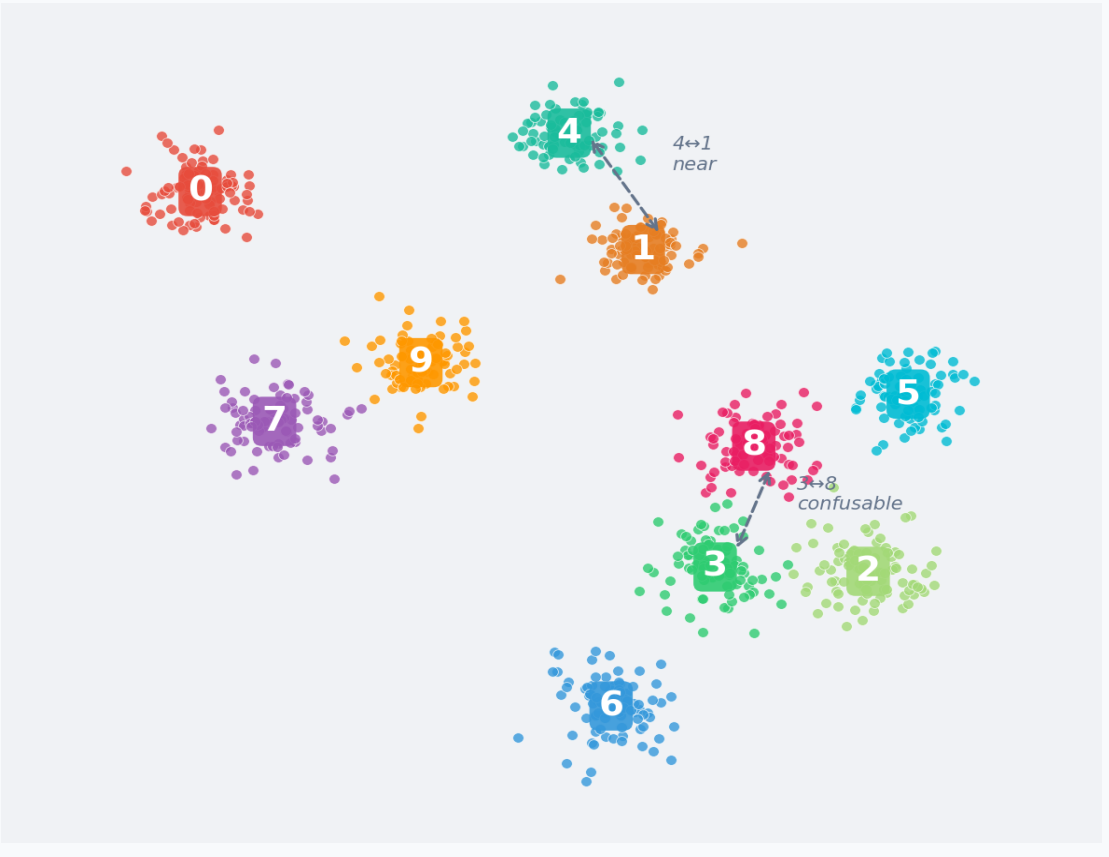
Important caveats

- Global distances are not trustworthy: far-apart clusters may not reflect true global geometry.
- Results are sensitive to perplexity and random seed, so reruns can change the picture.

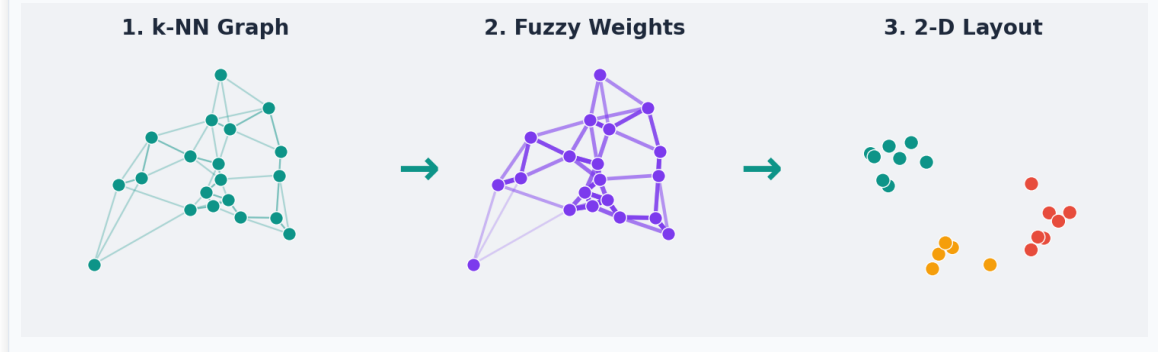


Uniform Manifold Approximation and Projection

UMAP on MNIST — CNN Last-Layer Features



How UMAP Works



Why UMAP Is the Modern Default



Massively Faster

Approximate k-NN graphs scale to millions of points. t-SNE is $O(n^2)$ naive, $O(n \log n)$ Barnes-Hut — UMAP handles 10–100× more data in the same time.



Better Global Structure

Cluster positions are more meaningful than in t-SNE. Relative distances between clusters tend to reflect actual feature-space proximity.



Reproducible & Tunable

More stable across reruns. Key params: `n_neighbors` (local vs global), `min_dist` (cluster tightness). Fewer surprises than t-SNE perplexity.



Practical tip: Use UMAP as your default first diagnostic — 5 lines of code on any learned representation (CNN, ViT, CLIP, VAE latent codes), colored by class or attribute. It immediately tells you whether the model has learned meaningful structure. Reserve t-SNE only when you need the tightest local clusters for a publication figure.

UMAP vs t-SNE

t-SNE

Best at

- Very clear local cluster visualization for exploratory analysis.

Trade-offs

- Slower on large datasets; global distances and relative cluster spacing can be misleading.

Use when

- You want a high-quality visual snapshot of local neighborhoods or class separation.

UMAP

Best at

- Faster embeddings, better scaling, and often more consistent structure across reruns.

Trade-offs

- Can still distort geometry; tuning choices can change the apparent structure.

Use when

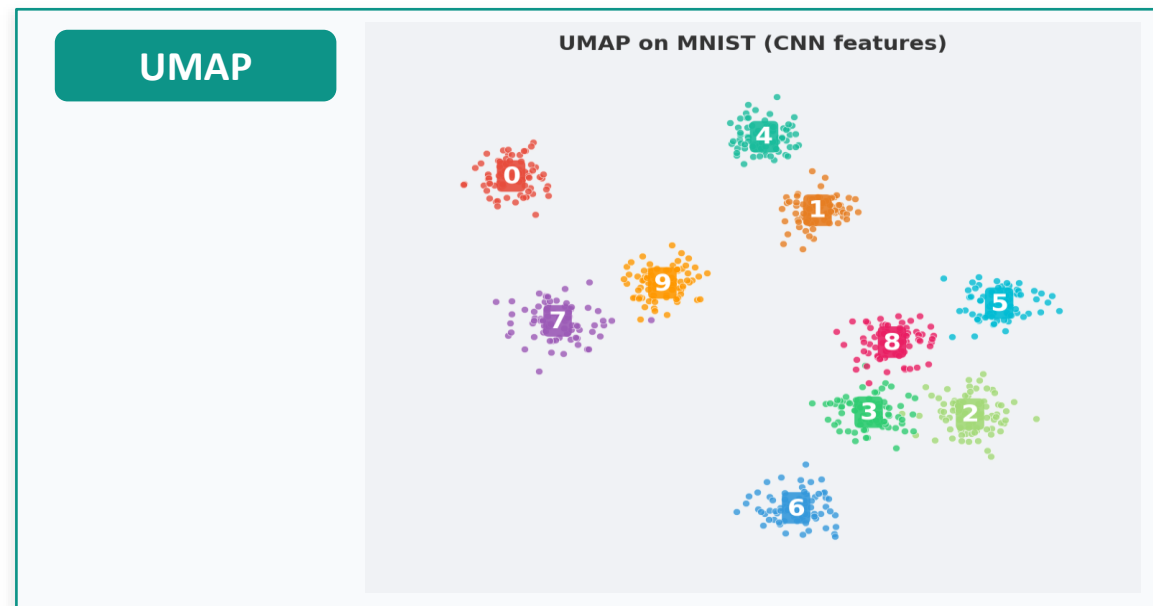
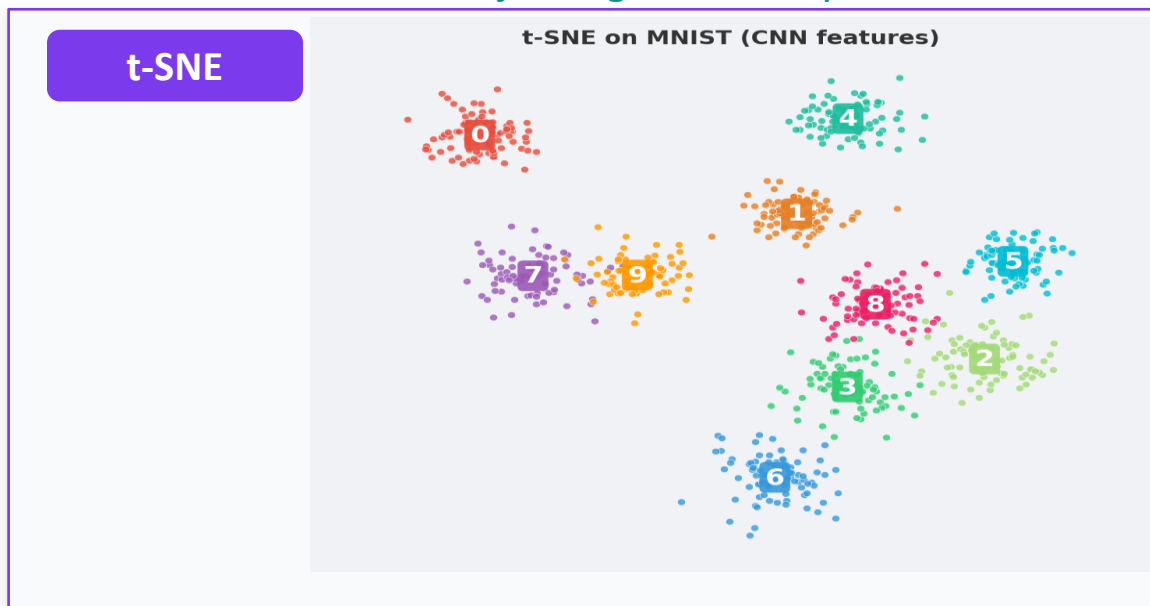
- You need larger datasets, repeated analyses, or a more reusable embedding pipeline.

Rule of thumb: t-SNE for local-cluster stories; UMAP for larger datasets and faster iteration.



Visualizing Learned Representations

t-SNE & UMAP — Projecting Feature Space to 2D



Pipeline: CNN / ViT / CLIP features → 2-D embedding (t-SNE or UMAP) → cluster visualization & diagnostics



Why UMAP over t-SNE?

4–10× faster — scales to millions
Better global structure preserved
More reproducible across reruns



Reading the Embedding

Tight clusters → good class separation
Neighbors → confusable classes (4↔9)
Outliers → noisy / mislabeled data



Important Caveats

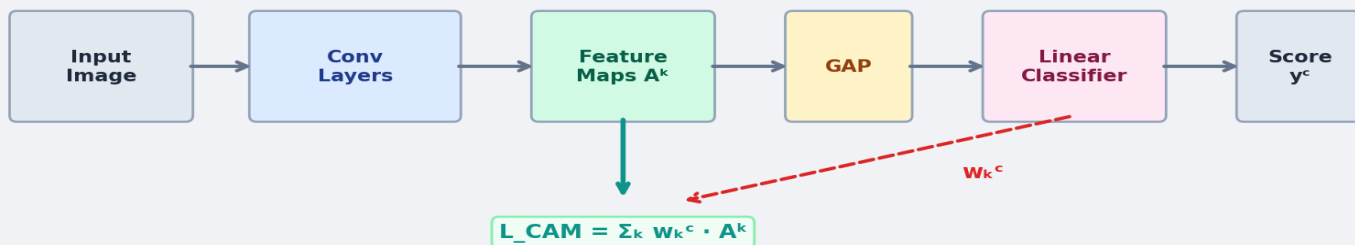
Global distances unreliable
Hyperparameter-sensitive (perplexity, n_neighbors)
Run multiple times before concluding

Rule of thumb: t-SNE for publication-quality local cluster figures. UMAP for everything else — faster, more scalable, more stable.

CAM: Class Activation Mapping

Visualizing Where the Network Looks

Architecture & Pipeline

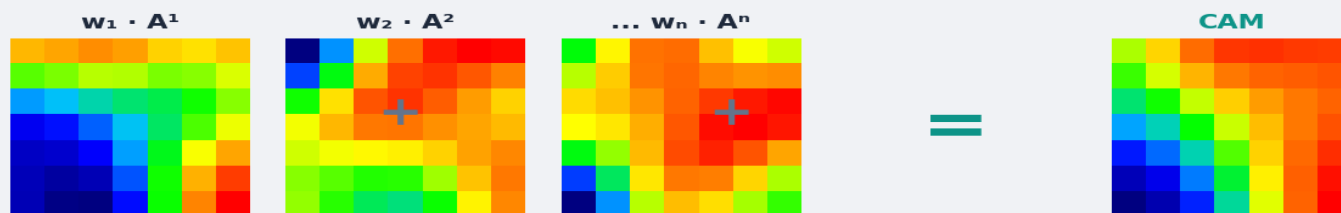


👁 How CAM Works

1. CNN with GAP + linear classifier
2. Keep last conv feature maps A^k
3. Read class weights w_k^c from classifier
4. Weighted sum: $L = \sum_k w_k^c \cdot A^k$
5. Upsample $\rightarrow$ overlay on image

No extra computation — just one matrix multiply!

Weighted Combination of Feature Maps



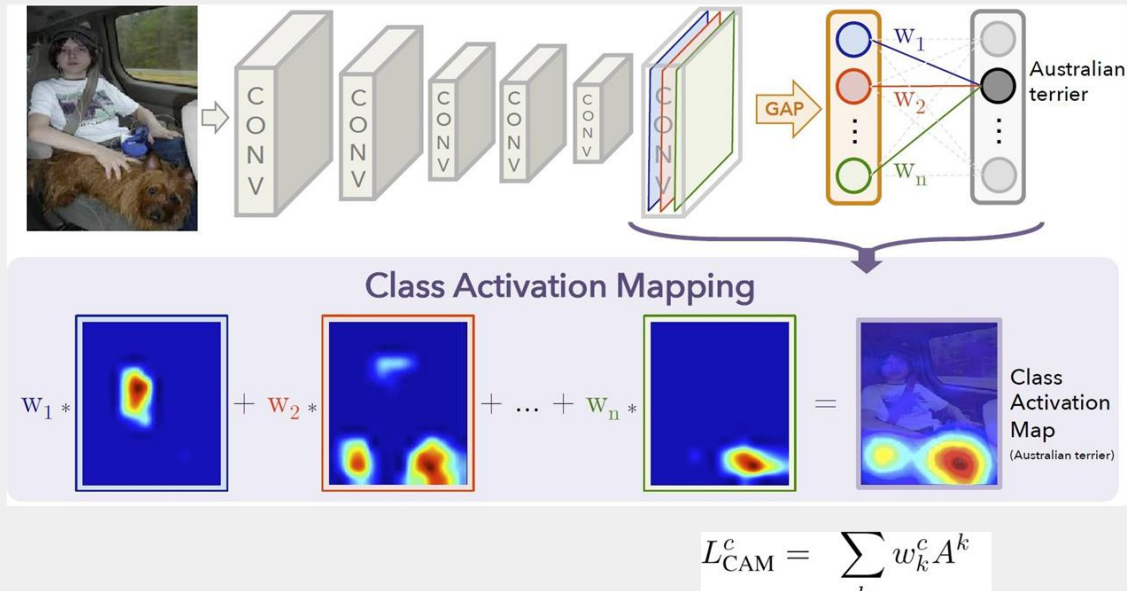
$$L^c = \sum_k w_k^c \cdot A^k$$

Each pixel = contribution to class score

✓ **Key insight:** GAP linearity lets you push class weights inside the spatial average — redistributing importance spatially across the feature maps.

⚠ **Limitation:** Requires GAP + linear classifier. Won't work with FC layers (VGG, AlexNet).

CAM: Class Activation Mapping



How CAM works

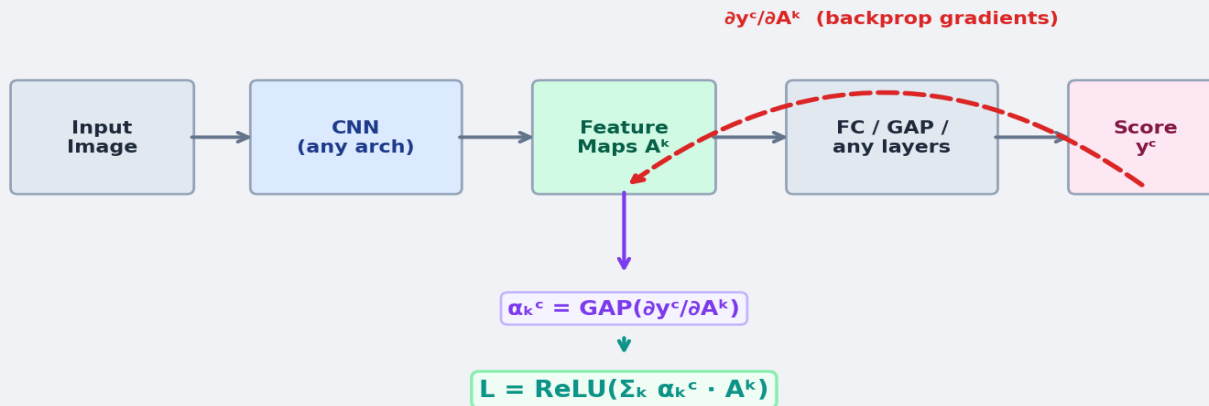
- Use a CNN that ends with global average pooling (GAP) and a linear classifier.
- Keep the last convolutional feature maps A_k .
- Read the class-specific classifier weights w_k^c .
- Combine feature maps with those weights: $L_{CAM}^c = \sum_k w_k^c A_k$.
- Upsample the class map to highlight image regions that support the class.

CAM requires GAP + linear classifier, so it is architecture-dependent.

Grad-CAM

Gradient-weighted Class Activation Mapping — Works with Any Architecture

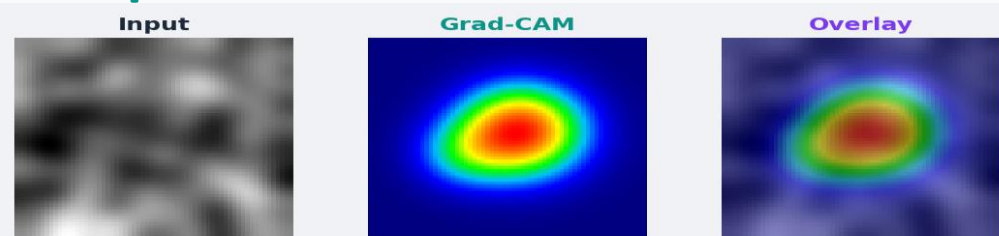
Forward Pass + Gradient Backprop



⚡ Step-by-Step

1. Forward pass $\rightarrow$ class score y^c
2. Backprop y^c to last conv layer $\rightarrow$ gradients $\partial y^c / \partial A^k$
3. GAP the gradients $\rightarrow$ channel weights α_k^c
4. Weighted sum: $\sum_k \alpha_k^c \cdot A^k$
5. Apply ReLU (positive evidence only)
6. Upsample $\rightarrow$ overlay heatmap

Example: Grad-CAM Visualization



$$\alpha_k^c = 1/Z \sum_{ij} \partial y^c / \partial A_{ij}^k \quad (\text{GAP of gradients})$$

$$L_{\text{Grad-CAM}} = \text{ReLU}(\sum_k \alpha_k^c \cdot A^k)$$

✂️ **Why Grad-CAM wins:** Architecture-agnostic (ResNet, VGG, ViT) • No retraining needed • Class-discriminative (different heatmaps per class) • For GAP+linear architectures, Grad-CAM = CAM mathematically

Sanity Checks for Explanations

Pretty Does Not Mean Faithful (Adebayo et al., 2018)



The Randomization Test

Idea

Randomize model weights (or shuffle training labels) and regenerate the saliency map with the same input image. Compare the two.

What should happen?

A faithful explanation should change dramatically. The randomized model has garbage weights — the heatmap should track the model, not the image.

Why it matters

Some popular methods (guided backprop, vanilla gradients) produce visually appealing heatmaps that barely change after randomization — they detect image structure, not model reasoning.



Faithful Method

- Heatmap changes when the model changes
- Evidence tracks class-specific reasoning
- Fails the test if it stays unchanged after randomization
- **Grad-CAM passes this test — heatmap changes substantially**



Suspicious Method

- Looks plausible but barely changes after randomization
- May reflect image priors (edges, contrast) rather than model reasoning
- **Pretty does not mean faithful**



The test: Trained model $\rightarrow$ heatmap₁ vs Randomized model $\rightarrow$ heatmap₂ $\rightarrow$ **If heatmap₁ $\approx$ heatmap₂, the method is NOT explaining the model**

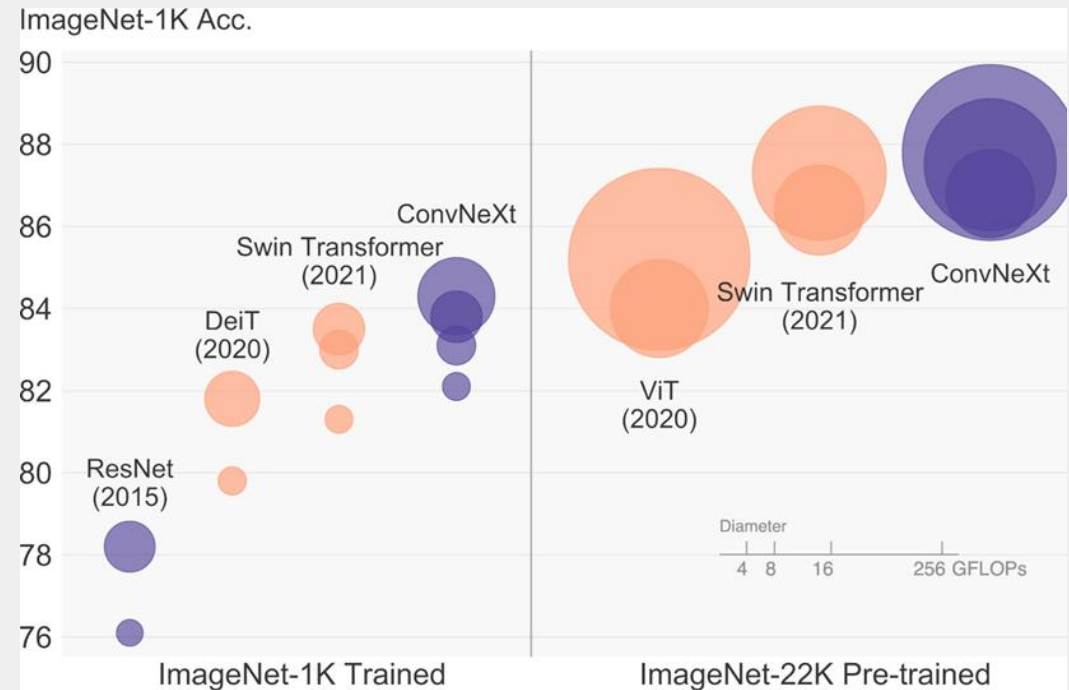


Bottom line: A good saliency map should fail when the model fails. If you randomize the model and the explanation still looks reasonable, it's explaining the image — not the model. Always sanity-check your visualization tools.

Are CNNs over?

Balanced view

- Research indicates CNNs are still very competitive, especially when latency, memory, or deployment cost matter.
- CNNs remain strong baselines: efficient, easier to train, and well understood in production pipelines.
- ViTs and hybrids can outperform at scale, but they often benefit from larger pretraining corpora and higher compute.



CNNs vs ViTs: CNNs often win on efficiency / latency and smaller-data regimes; ViTs often shine with large-scale pretraining and flexible global modeling.

When to Choose CNN vs ViT vs Hybrid

CNN

- Low latency or edge / mobile deployment.
- Smaller datasets or tighter compute budgets.
- Strong local inductive bias for textures and edges.
- Typical uses: robotics, medical baselines, embedded vision.

ViT

- Large-scale pretraining is available.
- Need stronger global context modeling.
- Aligns well with transformer tooling and foundation models.
- Typical uses: large image-text pretraining, research-scale models.

Hybrid

- Need CNN efficiency plus transformer-style context.
- Moderate data / compute budgets.
- Good compromise for detection or segmentation pipelines.
- Typical uses: Swin-like backbones, mixed-stage systems.

Pick the backbone for the deployment setting, not for fashion: data, latency, memory, and tooling all matter.



Selected references for the added / updated slides

- van der Maaten, L. & Hinton, G. (2008). Visualizing Data using t-SNE. JMLR.
- McInnes, L., Healy, J. & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv.
- Zhou, B. et al. (2016). Learning Deep Features for Discriminative Localization. CVPR.
- Selvaraju, R. R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV.
- Adebayo, J. et al. (2018). Sanity Checks for Saliency Maps. NeurIPS.
- Dosovitskiy, A. et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR.
- Liu, Z. et al. (2022). A ConvNet for the 2020s. CVPR.
- Uijlings, J. R. R. et al. (2013). Selective Search for Object Recognition. IJCV.
- Girshick, R. et al. (2014). Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. CVPR.
- Ren, S. et al. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS.
- Redmon, J. et al. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.
- Carion, N. et al. (2020). End-to-End Object Detection with Transformers. ECCV.
- Davis, S. & Mermelstein, P. (1980). Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences. IEEE TASSP.
- Park, D. S. et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech.

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a white, bold, sans-serif font.

HUST

Thanks